



清华大学
Tsinghua University

高性能计算导论

第7讲：处理器体系结构

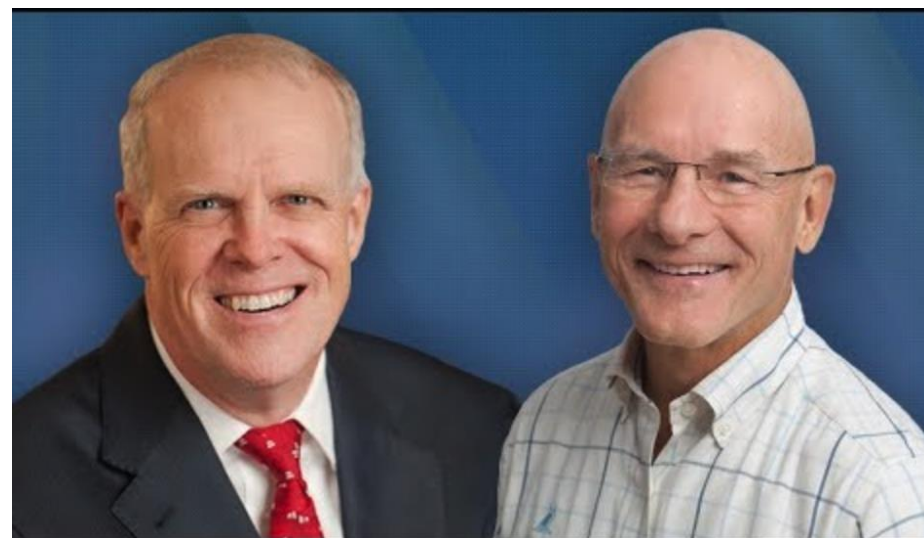
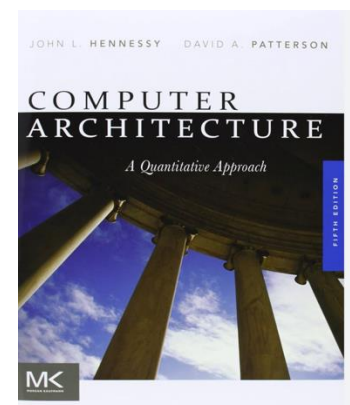
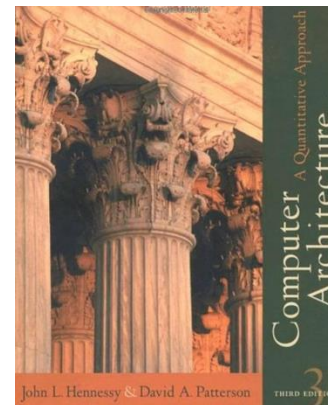
翟季冬
计算机系

目录

- 处理器基本概念
- 加速技术
 - 流水线
 - 超标量
 - 乱序执行
 - SIMD
 - 分支预测
- 存储层次
 - Cache
 - 虚存

2017 ACM Turing Award

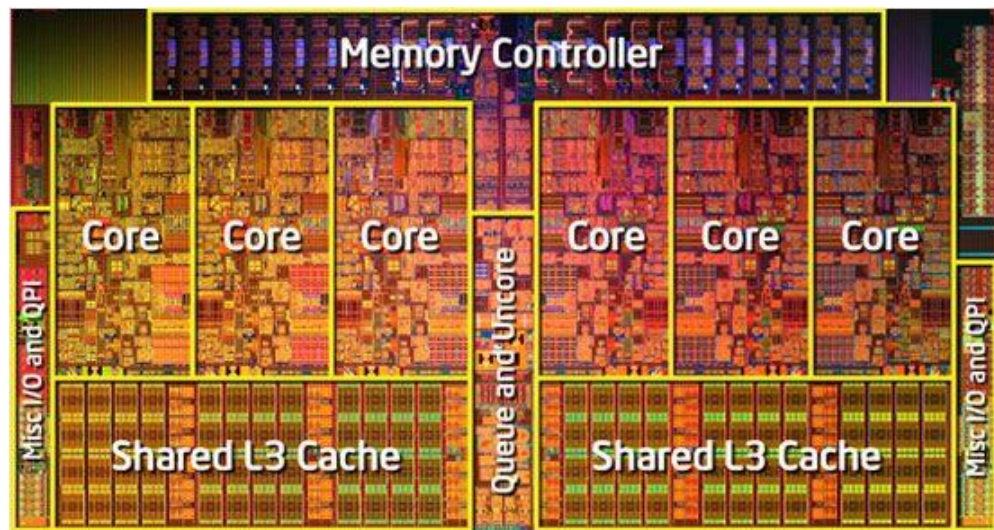
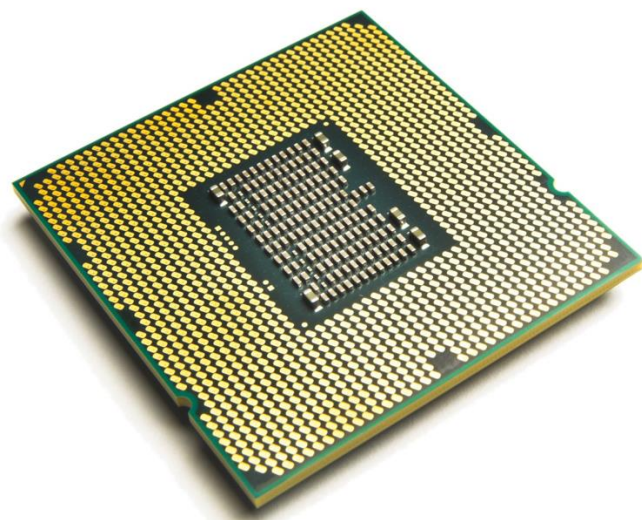
- John L. Hennessy (1952-)
- David A. Patterson (1947-)



For pioneering a systematic, quantitative approach to the design and evaluation of computer architectures with enduring impact on the microprocessor industry.

处理器

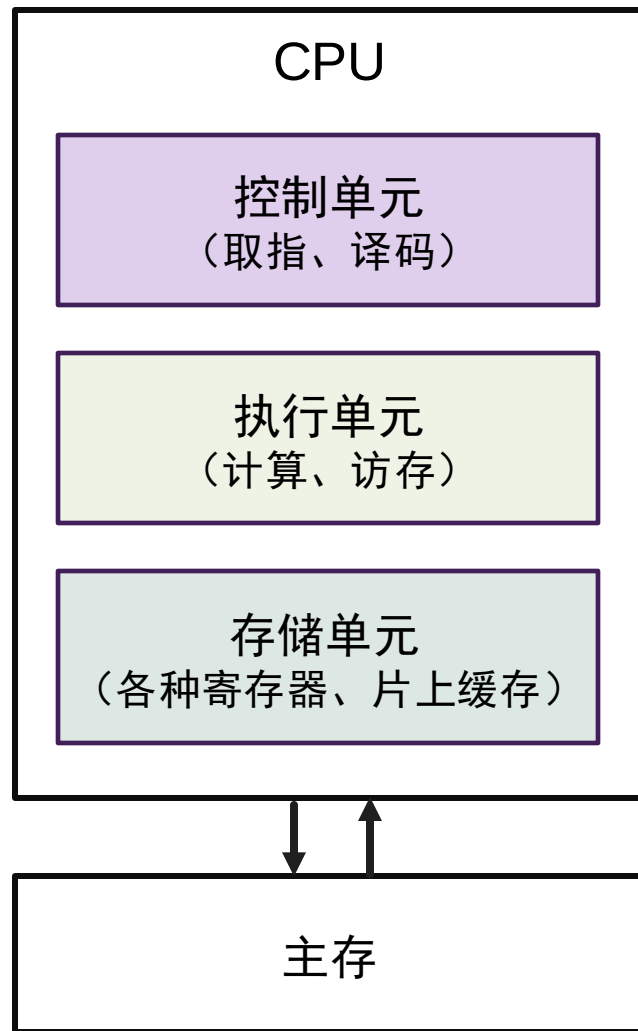
- 处理器 CPU



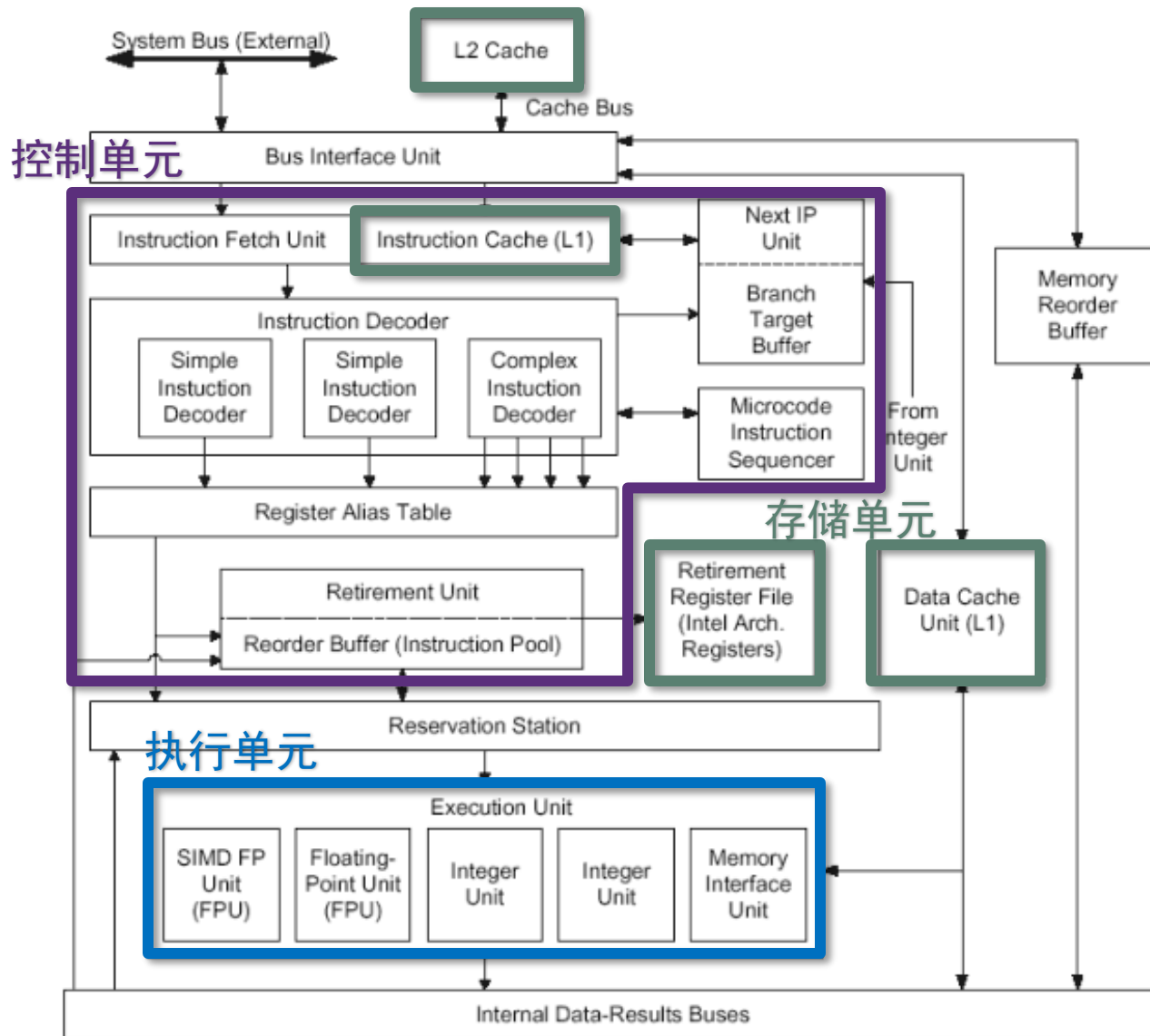
英特尔 i7-980X处理器

处理器基本概念：CPU的主要组成

- 控制单元 Control Unit (CU)
 - 负责从主存取得下一条要运行的指令（**取指**）
 - 并根据指令产生相应的控制信号（**译码**）
- 执行单元 Arithmetic/Logic Unit (ALU)
 - 执行整数/浮点/向量等操作（**计算**）
 - 进行内存读写（**访存**）
- 存储单元
 - **寄存器**
 - 程序计数器：当前指令地址
 - 指令寄存器：存储当前指令
 - 数据寄存器：存储数据
 - **各级片上缓存**：各级Cache



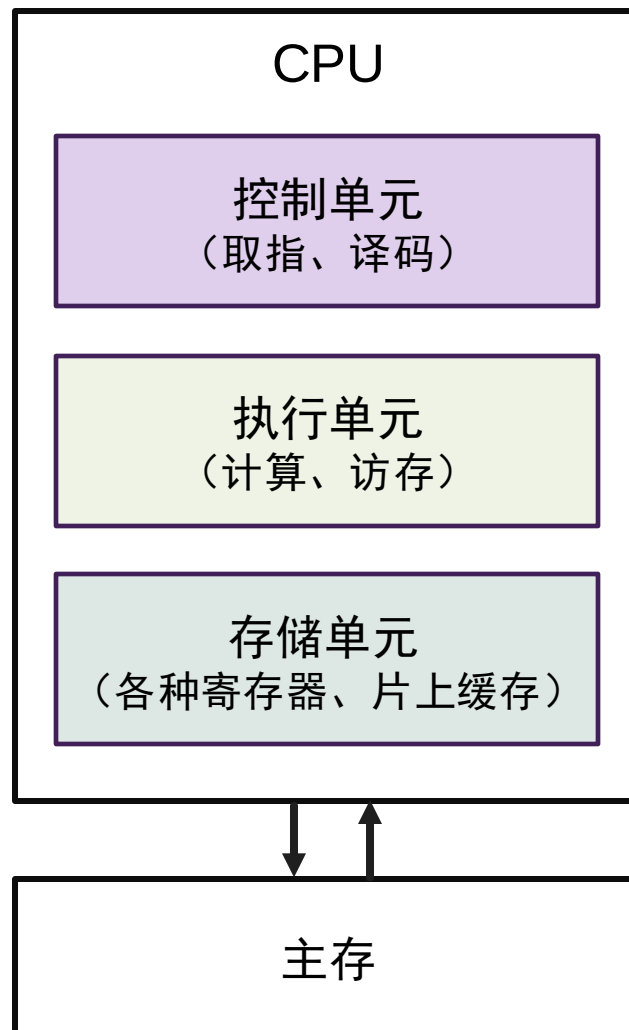
Intel Pentium 4



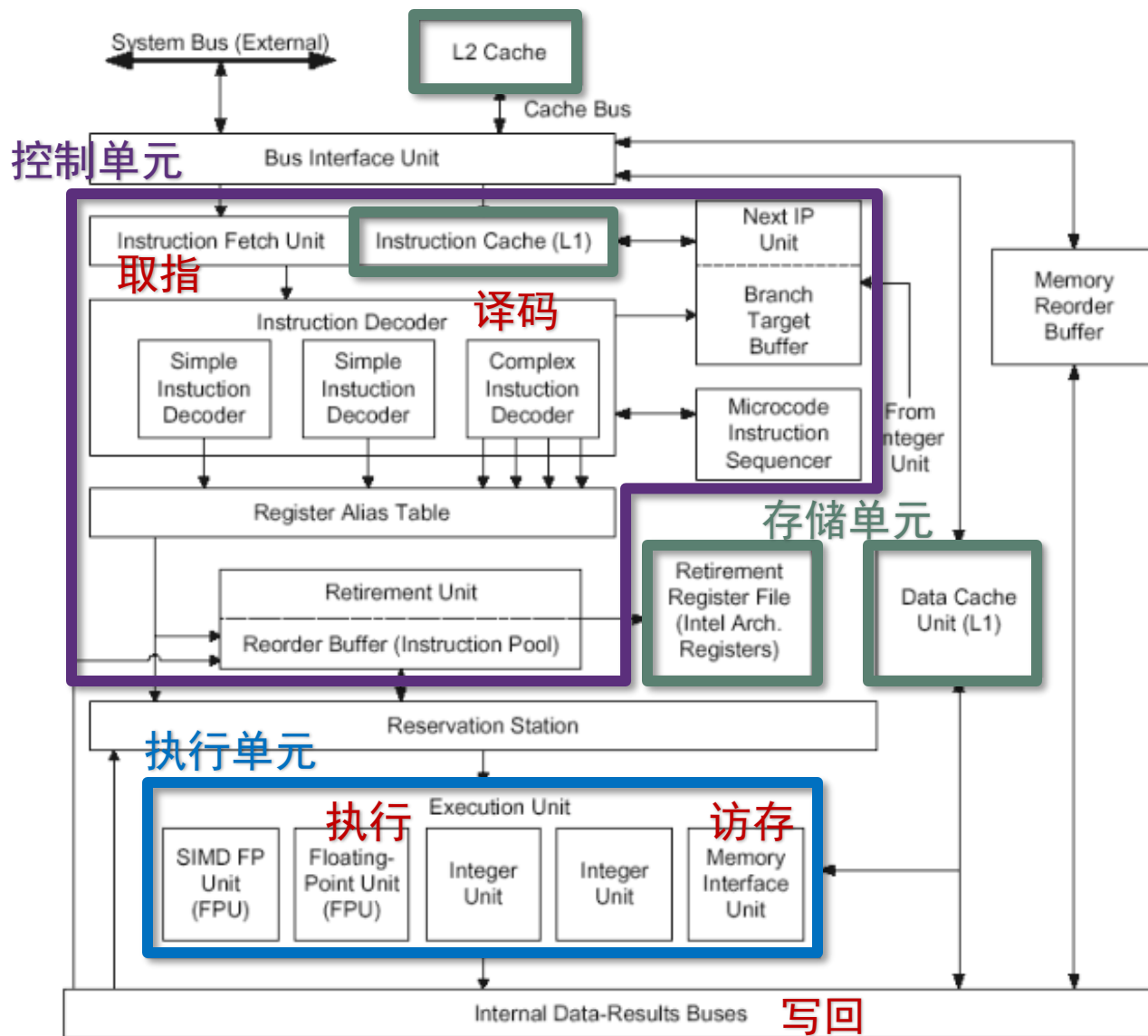
处理器基本概念：指令执行流程

■ 以MIPS的五阶段流程为例：

1. 取指（Instruction Fetch, IF）
 - 从主存取得需要执行的指令
2. 译码（Instruction Decode, ID）
 - 解析指令，产生相应的控制信号
3. 执行（Execute, EX）
 - 完成指令中与计算相关的操作
4. 访存（Memory, MEM）
 - 完成指令中与访问主存相关的操作
5. 写回（Write Back, WB）
 - 将结果写回寄存器或主存



Intel Pentium 4



处理器基本概念：周期与主频

■ 时钟周期 Clock Cycle

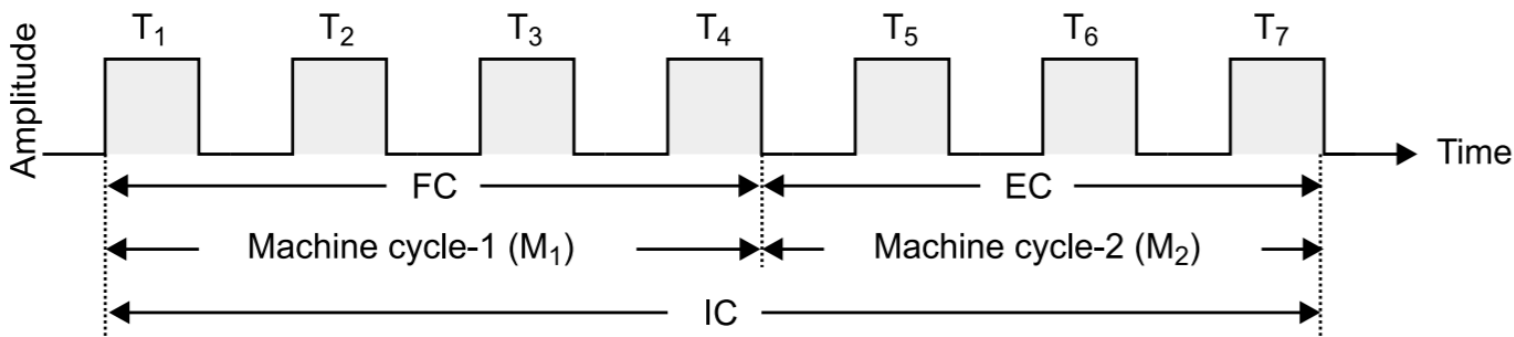
- 计算机中最基本、最小的时间单位
- 由实现CPU 电路的时钟产生
- 主频是时钟周期的倒数

■ 机器周期 Machine Cycle

- CPU完成一个基本操作（如取指，译码，执行等）所需要的时间

■ 指令周期 Instruction Cycle

- CPU完成一条指令（从取指到写回）所需要的时间



处理器基本概念：程序执行示例

■ 执行下面的程序

```
a = arr[0]
b = a + a
c = a * a
d = b + c
arr[0] = d
```

PC = 00

IF

ID

EX

MEM

WB

■ 对应的指令列表

地址	指令	备注
01	ld r0, &arr[0]	r0: a
02	add r1, r0, r0	r1: b
03	mul r2, r0, r0	r2: c
04	add r3, r1, r2	r3: d
05	st &arr[0], r3	

顺序执行

处理器基本概念：程序执行示例

■ 执行下面的程序

```
a = arr[0]
b = a + a
c = a * a
d = b + c
arr[0] = d
```

PC = 01	IF	ID	EX	MEM	WB
---------	----	----	----	-----	----

PC指向当前指令地址：01

当前阶段：IF

操作：从主存取得01地址对应的指令

■ 对应的指令列表

地址	指令	备注
01	ld r0, &arr[0]	r0: a
02	add r1, r0, r0	r1: b
03	mul r2, r0, r0	r2: c
04	add r3, r1, r2	r3: d
05	st &arr[0], r3	

顺序执行

处理器基本概念：程序执行示例

■ 执行下面的程序

```
a = arr[0]
b = a + a
c = a * a
d = b + c
arr[0] = d
```



PC指向当前指令地址：01

当前阶段：ID

操作：解析得知，需要访问主存数据 arr[0]，
后写回 r0 寄存器

■ 对应的指令列表

地址	指令	备注
01	ld r0, &arr[0]	r0: a
02	add r1, r0, r0	r1: b
03	mul r2, r0, r0	r2: c
04	add r3, r1, r2	r3: d
05	st &arr[0], r3	

顺序执行

处理器基本概念：程序执行示例

■ 执行下面的程序

```
a = arr[0]
b = a + a
c = a * a
d = b + c
arr[0] = d
```

PC = 01	IF	ID	EX	MEM	WB
---------	----	----	----	-----	----

PC指向当前指令地址：01

当前阶段：EX
操作：无需操作

■ 对应的指令列表

地址	指令	备注
01	ld r0, &arr[0]	r0: a
02	add r1, r0, r0	r1: b
03	mul r2, r0, r0	r2: c
04	add r3, r1, r2	r3: d
05	st &arr[0], r3	

顺序执行

处理器基本概念：程序执行示例

■ 执行下面的程序

```
a = arr[0]
b = a + a
c = a * a
d = b + c
arr[0] = d
```

PC = 01	IF	ID	EX	MEM	WB
---------	----	----	----	-----	----

PC指向当前指令地址：01

当前阶段：MEM

操作：访问主存并取得arr[0]

■ 对应的指令列表

地址	指令	备注
01	ld r0, &arr[0]	r0: a
02	add r1, r0, r0	r1: b
03	mul r2, r0, r0	r2: c
04	add r3, r1, r2	r3: d
05	st &arr[0], r3	

顺序执行

处理器基本概念：程序执行示例

■ 执行下面的程序

```
a = arr[0]
b = a + a
c = a * a
d = b + c
arr[0] = d
```

PC = 01	IF	ID	EX	MEM	WB
---------	----	----	----	-----	----

PC指向当前指令地址：01

当前阶段：WB

操作：将arr[0]写入 r0 寄存器

■ 对应的指令列表

地址	指令	备注
01	ld r0, &arr[0]	r0: a
02	add r1, r0, r0	r1: b
03	mul r2, r0, r0	r2: c
04	add r3, r1, r2	r3: d
05	st &arr[0], r3	

顺序执行

处理器基本概念：程序执行示例

■ 执行下面的程序

```
a = arr[0]
b = a + a
c = a * a
d = b + c
arr[0] = d
```



PC指向当前指令地址：02

顺序执行下一条指令，具体执行阶段为EX

■ 对应的指令列表

地址	指令	备注
01	ld r0, &arr[0]	r0: a
02	add r1, r0, r0	r1: b
03	mul r2, r0, r0	r2: c
04	add r3, r1, r2	r3: d
05	st &arr[0], r3	

顺序执行

| 这个例子存在什么性能问题？

这个例子存在什么性能问题？

指令1的MEM阶段

■ 执行下面的程序

```
a = arr[0]
b = a + a
c = a * a
d = b + c
arr[0] = d
```



PC指向当前指令地址：01

当前阶段：MEM

操作：访问主存并取得arr[0]

CPU单元闲置

■ 对应的指令列表

地址	指令	备注
01	ld r0, &arr[0]	r0: a
02	add r1, r0, r0	r1: b
03	mul r2, r0, r0	r2: c
04	add r3, r1, r2	r3: d
05	st &arr[0], r3	

顺序执行

这个例子存在什么性能问题？

指令2的执行

■ 执行下面的程序

```
a = arr[0]
b = a + a
c = a * a
d = b + c
arr[0] = d
```

PC = 02	IF	ID	EX	MEM	WB
---------	----	----	----	-----	----

PC指向当前指令地址：02

顺序执行下一条指令，具体执行阶段为EX

■ 对应的指令列表

地址	指令	备注
01	ld r0, &arr[0]	r0: a
02	add r1, r0, r0	r1: b
03	mul r2, r0, r0	r2: c
04	add r3, r1, r2	r3: d
05	st &arr[0], r3	

这两条指令可以并行执行

顺序执行（一条指令）
无法挖掘指令并行性

这个例子存在什么性能问题？

- 数组的每个元素都要进行操作

```
for i in 1...n:  
    a = arr[i]  
    b = a + a  
    c = a * a  
    d = b + c  
    arr[i] = d
```

依次操作每个元素

无法挖掘数据并行性

CPU 加速技术

- 刚才例子的问题
 - CPU单元闲置
- 顺序执行一条指令无法挖掘指令并行性
- 依次操作每个元素无法挖掘数据并行性

CPU 加速技术

- 刚才例子的问题与解决方案
 - CPU单元闲置
 - 流水线：多条指令以流水线模式执行
 - 顺序执行一条指令无法挖掘指令并行性
 - 超标量：同时执行多条相邻的指令
 - 乱序执行：允许后面的指令先于前面的指令执行
 - 依次操作每个元素无法挖掘数据并行性
 - 单指令多数据，SIMD：单一指令操作多个数据

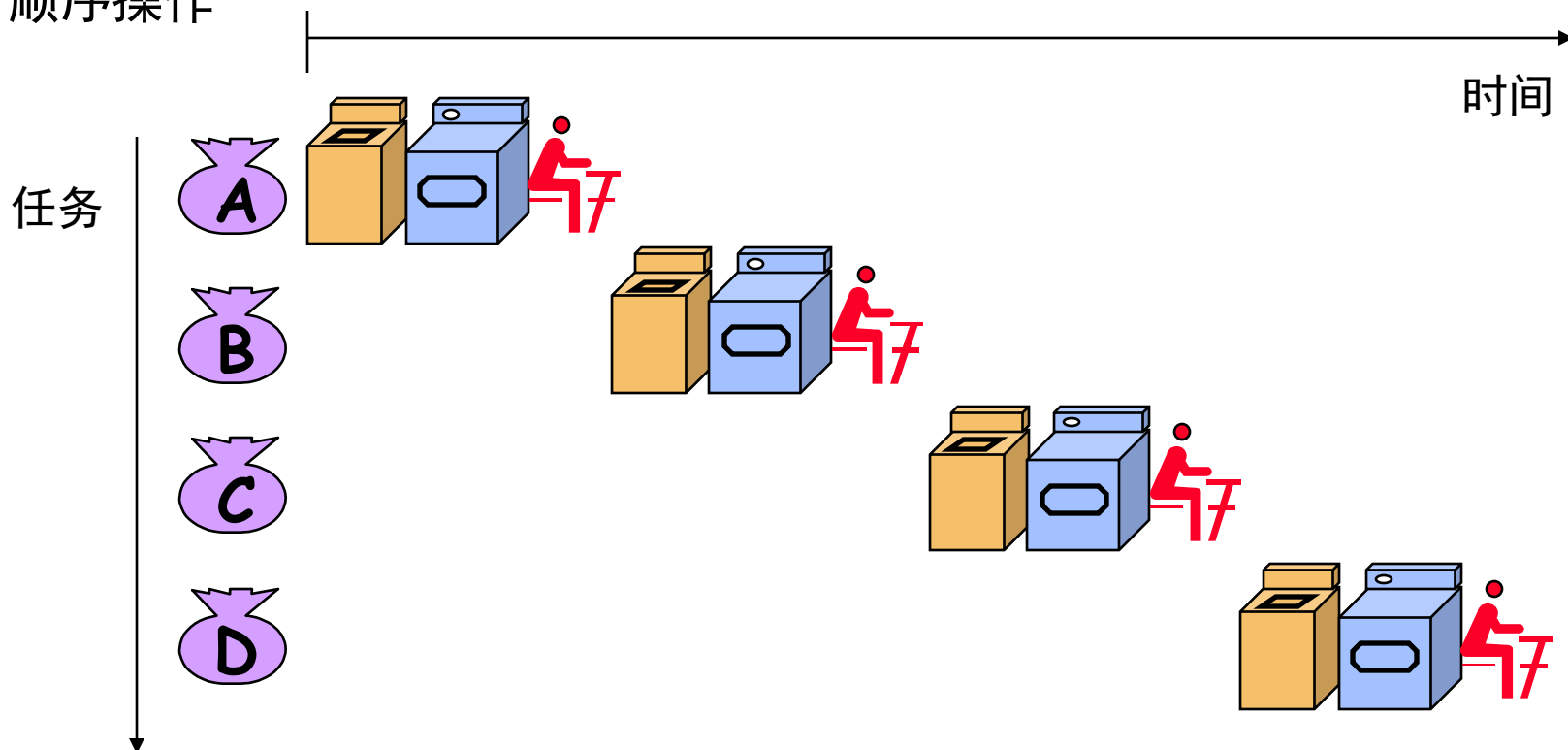
流水线

加速技术：流水线

■ 以洗衣服为例

- **流程**：洗衣服（30分钟）、烘干（40分钟）、收纳（20分钟）
- **顺序执行**：4袋衣服：4 * 90分钟 = **6小时**（硬件有idle、资源浪费）

■ 顺序操作

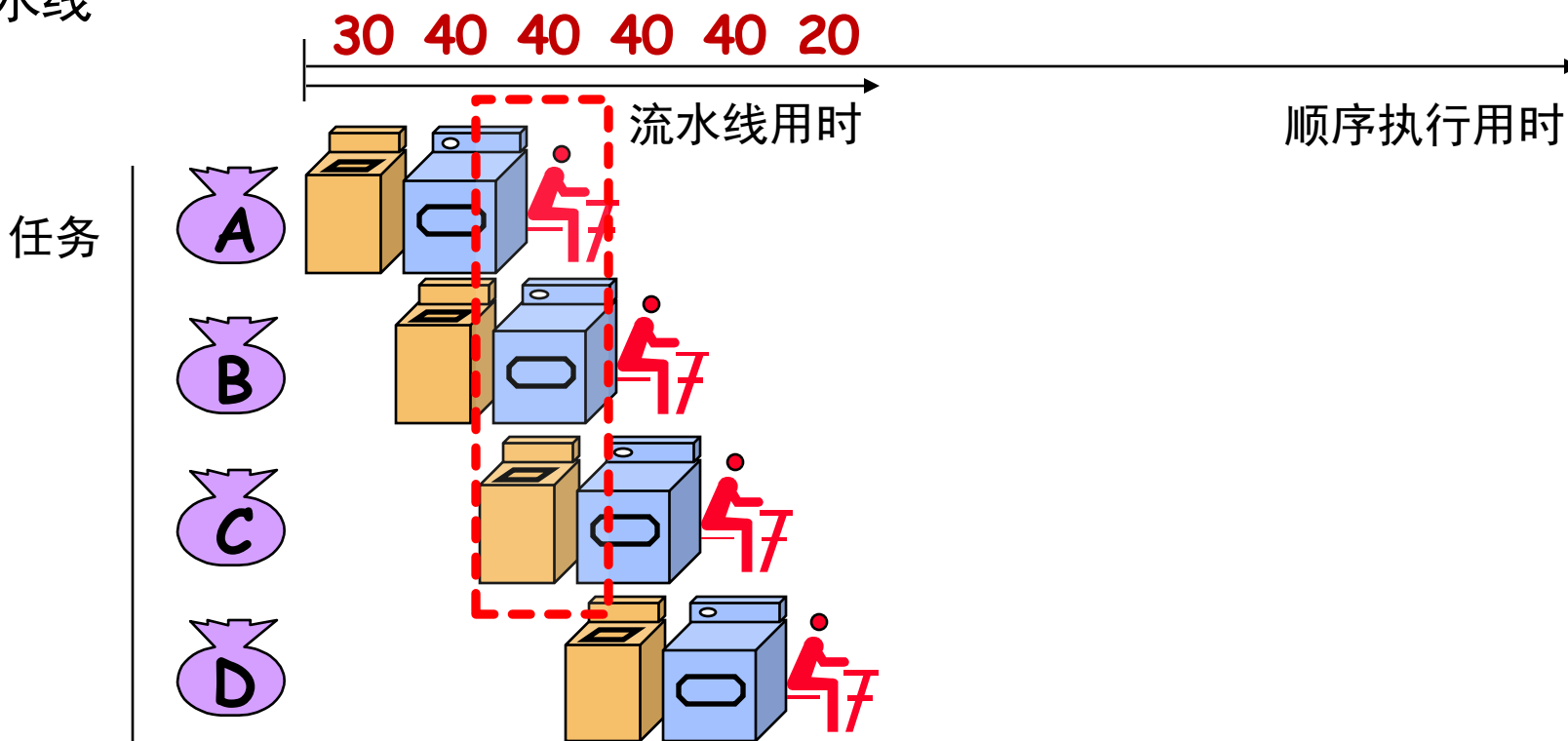


加速技术：流水线

■ 以洗衣服为例

- **流程**：洗衣服（30分钟）、烘干（40分钟）、收纳（20分钟）
- **流水线执行**：4袋衣服 $30 + 40 * 4 + 20 = 3.5$ 小时 vs. 顺序 6 小时
- 所有硬件几乎一直被占用，利用率高

■ 流水线

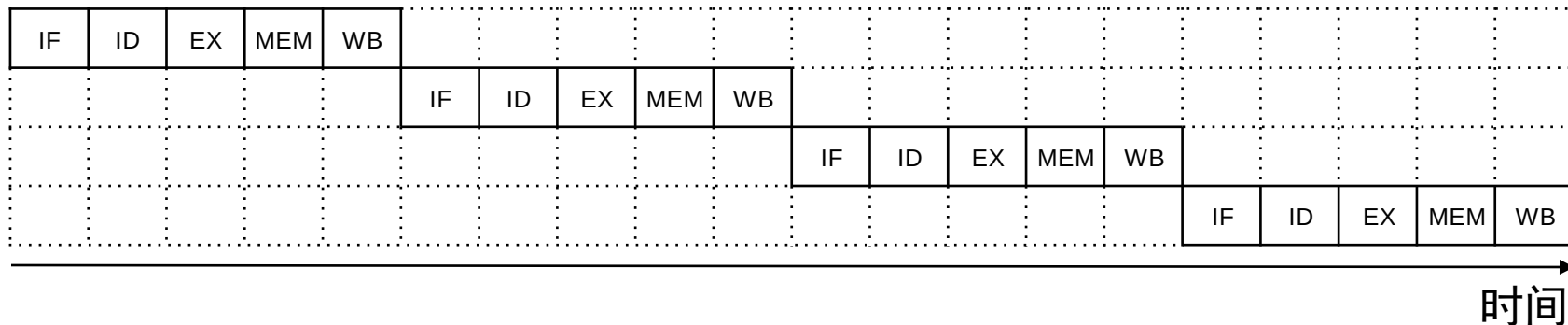


工厂流水线



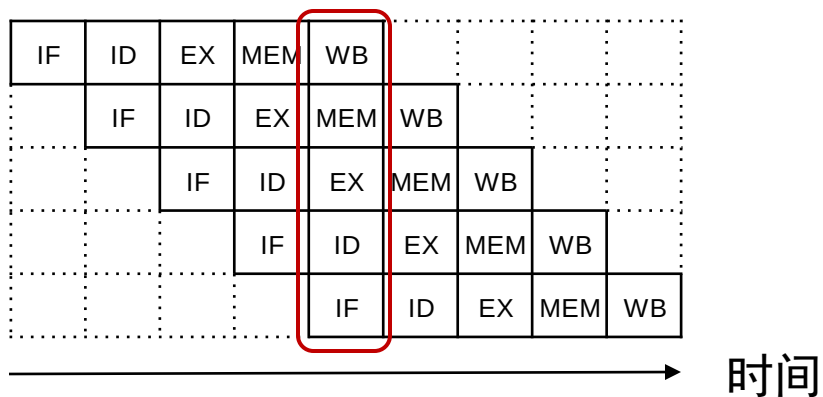
加速技术：流水线

■ 指令的顺序执行：



■ 指令的流水线执行：

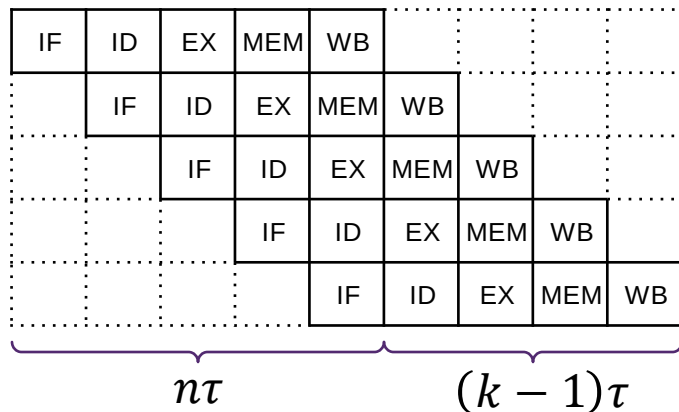
- 对于一条指令而言：顺序执行每个阶段
- 对于多条指令而言：并发执行
- 对于处理单元而言：（理想情况下）同一时刻没有闲置的处理单元



加速技术：流水线

流水线的延迟：完成一条指令所用时间

- 流水线并不改变指令延迟，每条指令执行所用时间与顺序执行相同



流水线的带宽（吞吐量）：每个周期完成指令的个数

- 记 n 条指令完成总时间数为 T_n ，带宽为 $\frac{n}{T_n}$

- 假设一条指令分为 k 个阶段完成，每个阶段时间为 τ ，则

$$T_n = n\tau + (k - 1)\tau = (n + k - 1)\tau$$

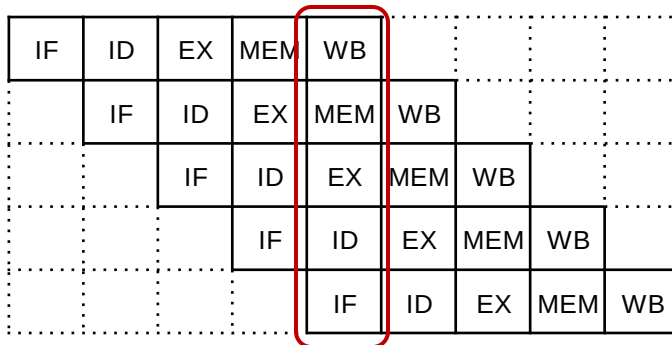
- 相对于顺序执行的加速比

$$\frac{nk\tau}{(n + k - 1)\tau} = \frac{nk}{n + k - 1} = \frac{k}{1 + \frac{k - 1}{n}} \xrightarrow{n \rightarrow \infty} k$$

加速技术：流水线

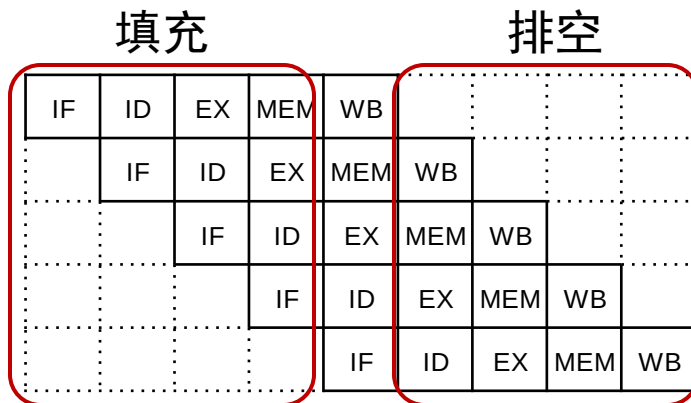
■ 影响流水线带宽的因素

- **阶段划分不均衡**：流水线受制于**最慢的阶段**
 - 假设MEM阶段是其他执行时间的2倍
 - 流水线按照节拍执行，其他阶段也必须被拉长到2倍时间



■ **流水线填充与排空：**

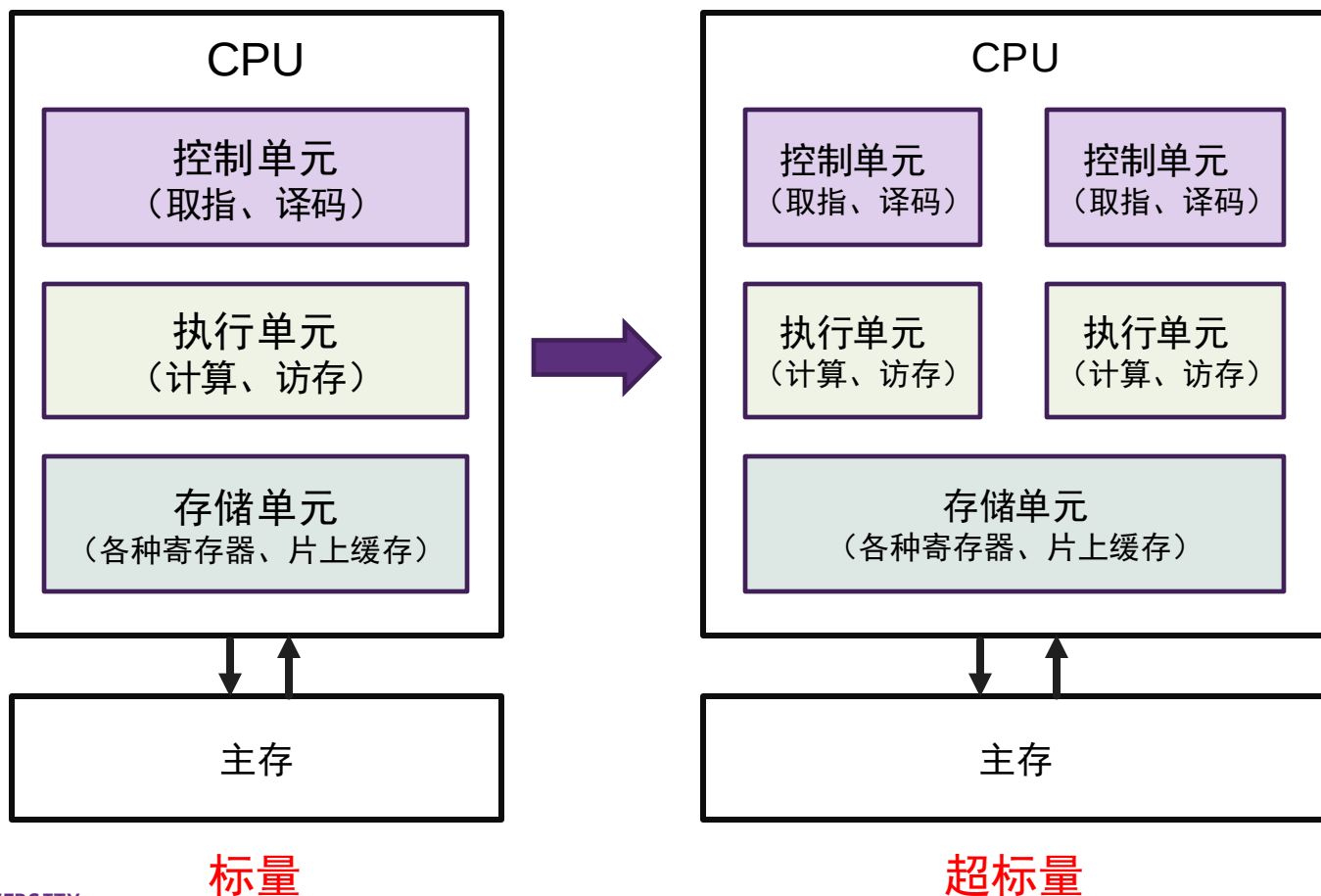
- 存在处理单元闲置



超标量

加速技术：超标量

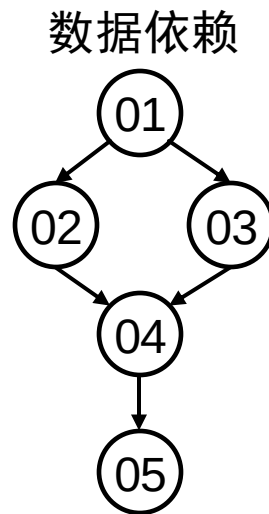
- 通过**增加硬件单元**同时执行多条指令（**流水线没有增加硬件单元**）
 - **MIMD**：多指令流、多数据流
 - ILP（Instruction Level Parallelism）：**指令级并行**



加速技术：超标量

- 通过增加硬件单元**同时执行多条指令**
- 例如：顺序执行的超标量处理器（假设有 2 个计算单元）
 - 一次可以执行两个指令

地址	指令
01	ld r0, &arr[0]
02	add r1, r0, r0
03	mul r2, r0, r0
04	add r3, r1, r2
05	st &arr[0], r3



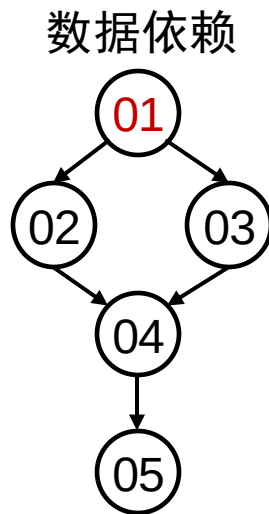
加速技术：超标量

- 通过增加硬件单元同时执行多条指令
- 例如：顺序执行的超标量处理器（假设有 2 个计算单元）
 - 一次可以执行两个指令

检查01、02：

二者存在依赖，故只可执行01

地址	指令
01	ld r0, &arr[0]
02	add r1, r0, r0
03	mul r2, r0, r0
04	add r3, r1, r2
05	st &arr[0], r3



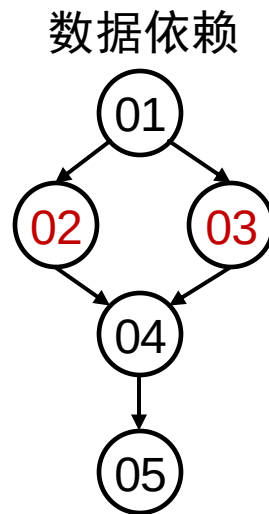
加速技术：超标量

- 通过增加硬件单元同时执行多条指令
- 例如：顺序执行的超标量处理器（假设有 2 个计算单元）
 - 一次可以执行两个指令

执行01后，检查02、03：

可同时执行

地址	指令
01	ld r0, &arr[0]
02	add r1, r0, r0
03	mul r2, r0, r0
04	add r3, r1, r2
05	st &arr[0], r3



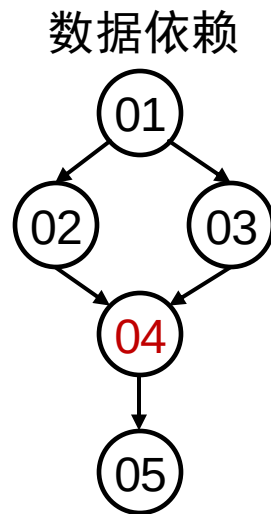
加速技术：超标量

- 通过增加硬件单元同时执行多条指令
- 例如：顺序执行的超标量处理器（假设有 2 个计算单元）
 - 一次可以执行两个指令

执行02、03后，

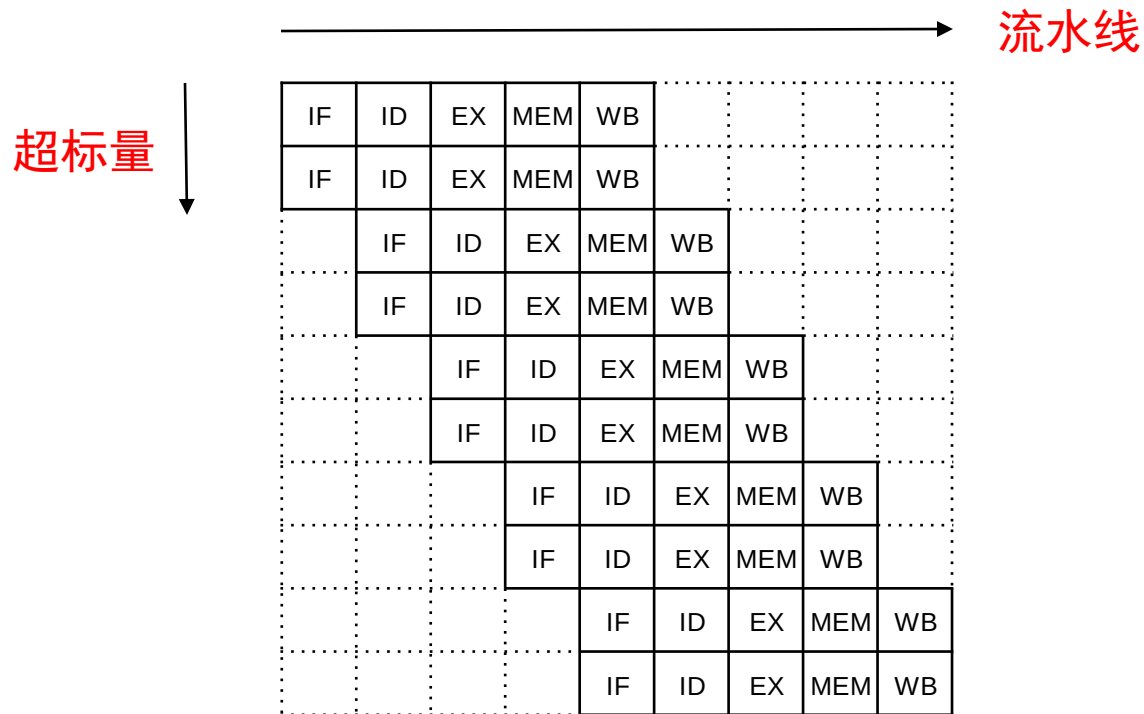
04、05需要顺序执行

地址	指令
01	ld r0, &arr[0]
02	add r1, r0, r0
03	mul r2, r0, r0
04	add r3, r1, r2
05	st &arr[0], r3

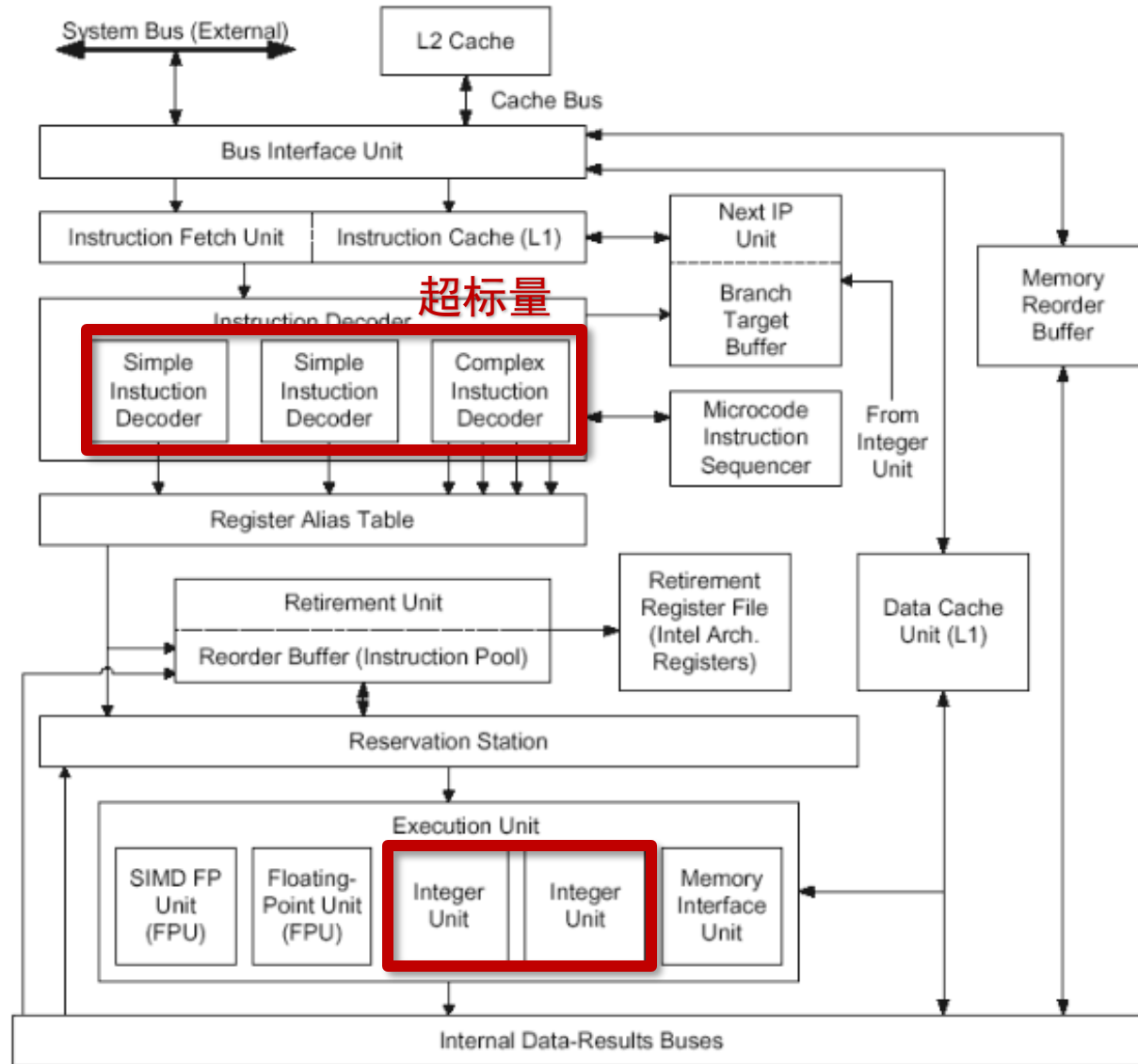


加速技术：超标量和流水线

- 超标量与流水线结合：
 - 进一步提高处理器吞吐量



Intel Pentium 4



乱序执行

顺序执行 vs. 乱序执行

- 打破原始程序指令排列顺序，允许后面指令先于前面指令执行

阶段	顺序执行 in-order
指令获取	获取指令
解析指令	✓ 如果该指令数据就绪，指令就被发送到合适的功能单元 ✓ 否则，处理器会等待数据
指令执行	指令在合适功能单元中被执行
结果写回	运算结果被写回寄存器

顺序执行 vs. 乱序执行

- 打破原始程序指令排列顺序，允许后面指令先于前面指令执行

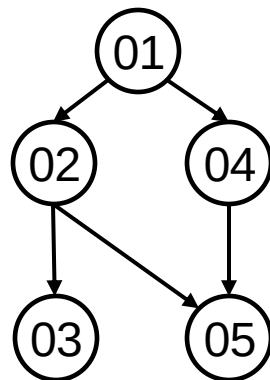
阶段	顺序执行 in-order	乱序执行 out-of-order
指令获取	获取指令	获取指令
解析指令	✓ 如果该指令数据就绪，指令就被发送到合适的功能单元 ✓ 否则，处理器会等待数据	✓ 指令被发送到一个缓冲区 ✓ 数据就绪指令离开缓冲区（乱序）、未就绪指令继续等待
指令执行	指令在合适功能单元中被执行	指令在合适功能单元中被执行
结果写回	运算结果被写回寄存器	✓ 运算结果被放到一个序列 ✓ 仅当该指令之前所有指令已经将它们结果写回寄存器后，该指令的结果才会被写回寄存器

加速技术：乱序执行

- 打破原始程序指令排列顺序，允许后面的指令先于前面的指令执行
- 考虑如下程序

地址	程序
01	<code>a = arr[0]</code>
02	<code>b = a + a</code>
03	<code>c = b * b</code>
04	<code>d = a * a</code>
05	<code>e = d + b</code>

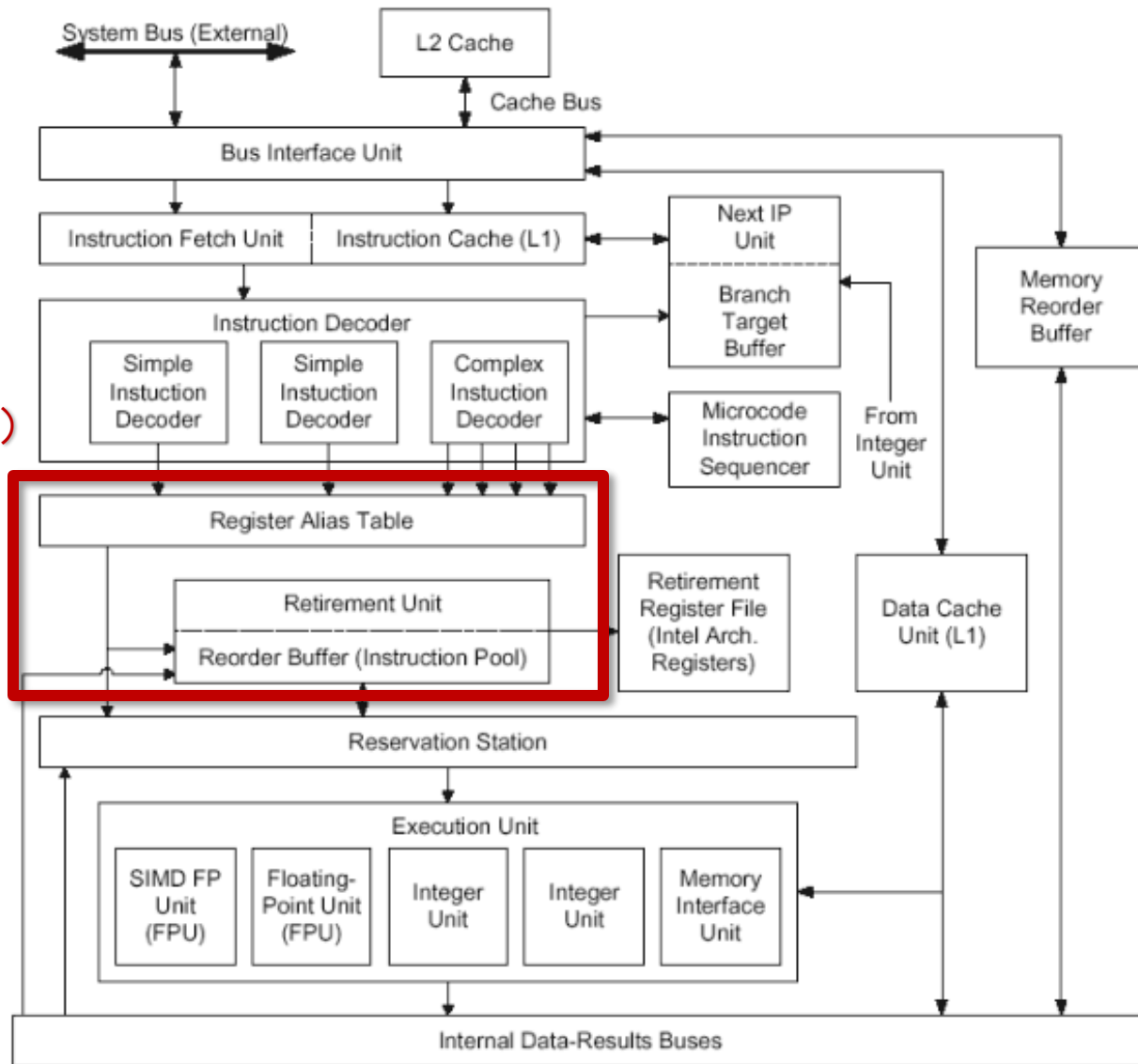
指令依赖关系



- **顺序执行**的超标量处理器
 - $\{01\} \rightarrow \{02\} \rightarrow \{03, 04\} \rightarrow \{05\}$
- **乱序执行**的超标量处理器
 - $\{01\} \rightarrow \{02, 04\} \rightarrow \{03, 05\}$
- 现代处理器通过**寄存器重命名**技术进一步挖掘乱序执行的潜力
- 乱序执行需要**复杂电路**，和低功耗设计违背

Intel Pentium 4

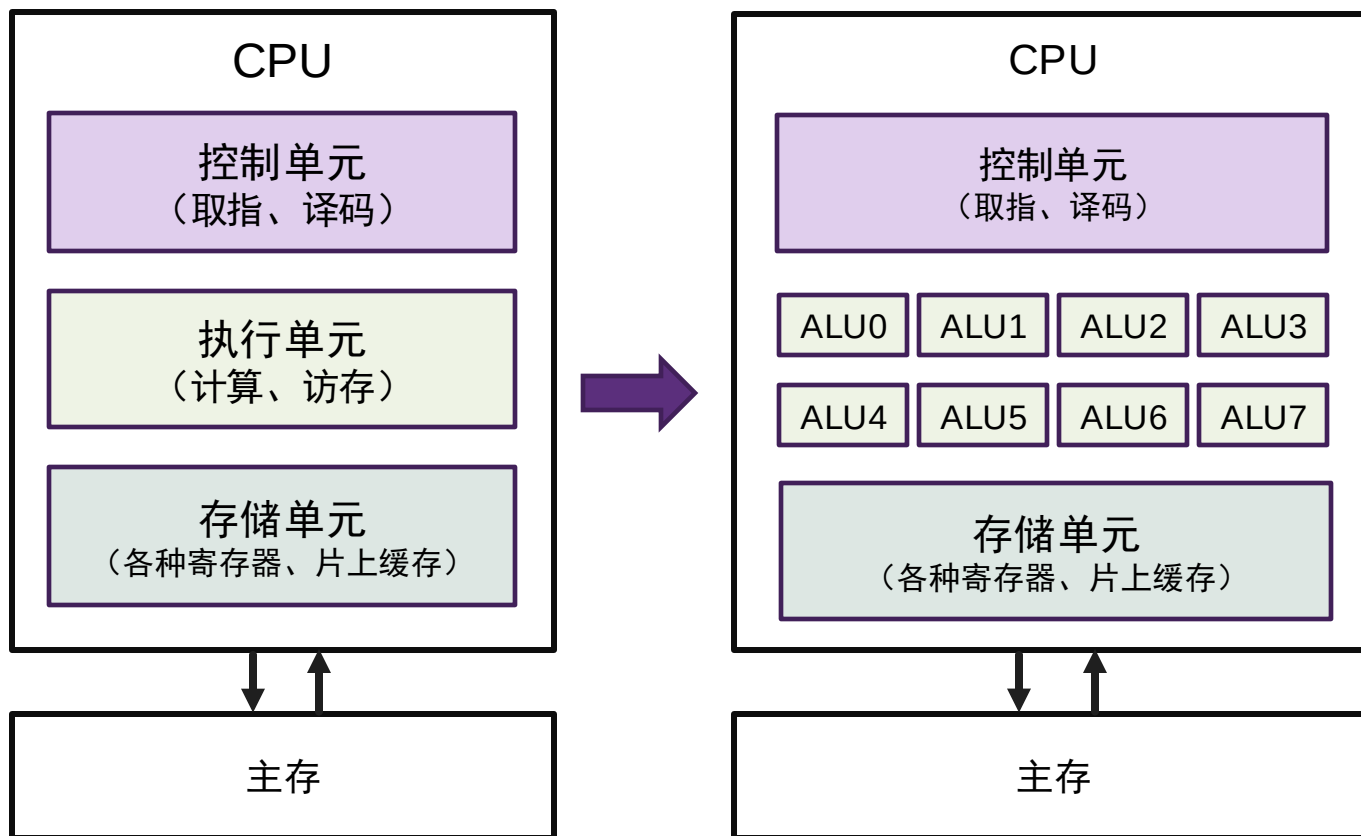
乱序执行
(寄存器重命名)



向量处理

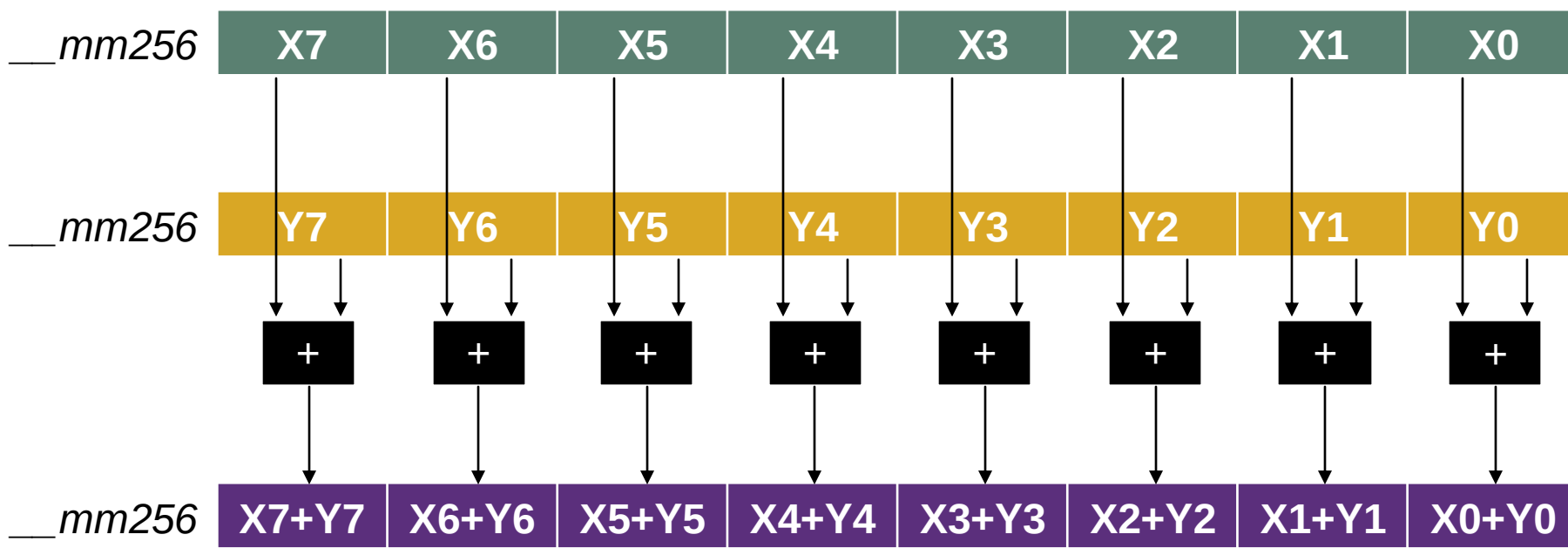
加速技术：SIMD

- SIMD - Single Instruction, Multiple Data
 - 单指令流、多数据流
 - 所有 ALU 执行同一条指令、每个 ALU 处理不同的数据

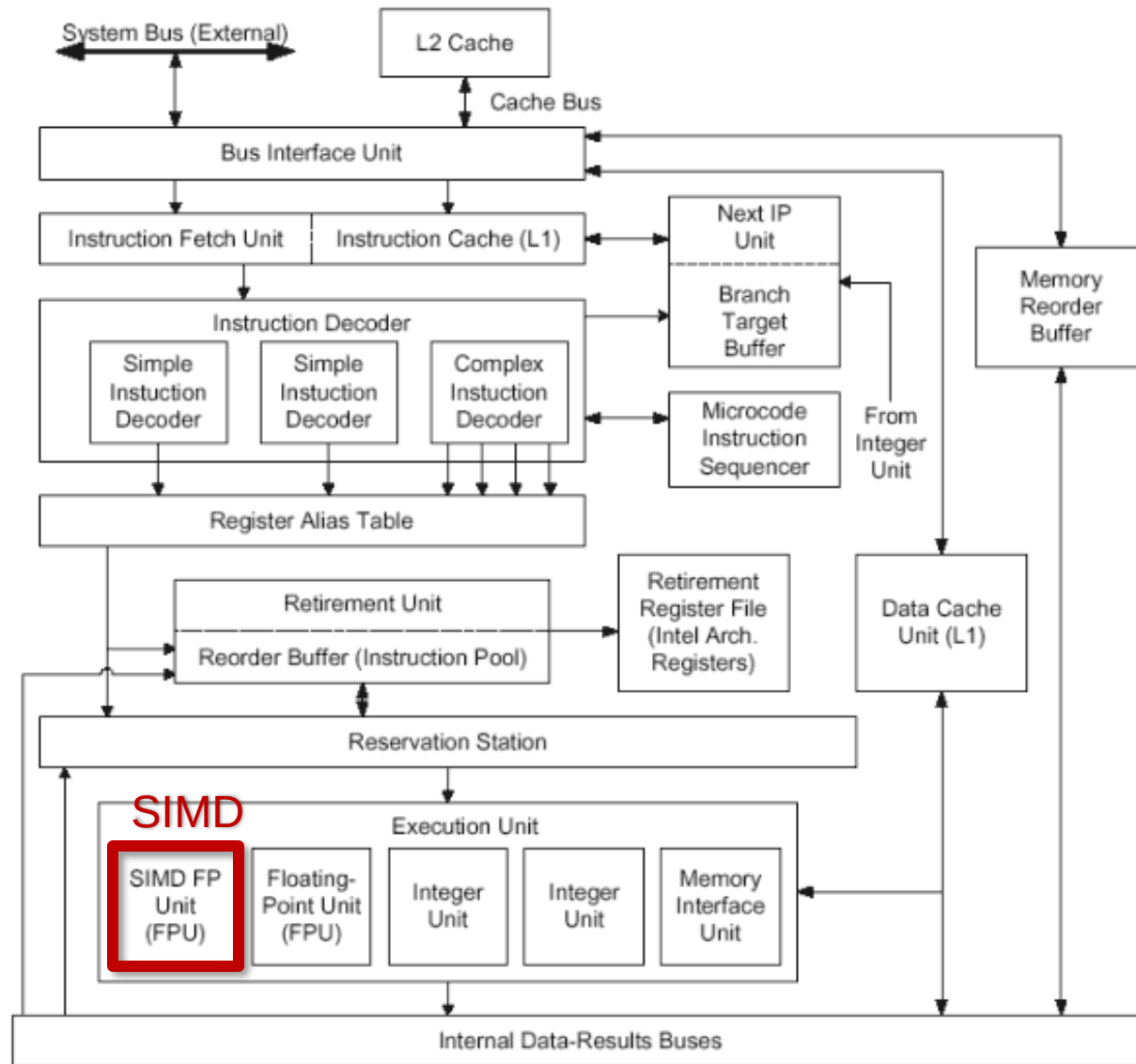


加速技术：SIMD

- 与之前技术不同，SIMD需要使用特殊指令实现
- 示例：Intel CPU上的向量扩展指令
 - 加法指令示例：AVX
 - `_mm256_add_epi32`（8个整数同时执行add）



Intel Pentium 4



分支预测

程序分支

- 如果程序有分支，流水线如何加载指令？
 - 流水线核心需要有大量无依赖的指令

```
if (x > 0)
    temp *= kValue;
else
    temp *= gValue;
```

IF	ID	EX	MEM	WB				
	IF	ID	EX	MEM	WB			
		IF	ID	EX	MEM	WB		
			IF	ID	EX	MEM	WB	
				IF	ID	EX	MEM	WB

加速技术：分支预测

■ 条件跳转会使流水线失效

- 当遇到条件跳转时，处理器无法得知需要将哪一条后续指令读入流水线
- 由于无法提前获知分支跳转目标，会造成：
 - 流水线停顿 pipeline stalled
 - 超标量、乱序执行受到影响

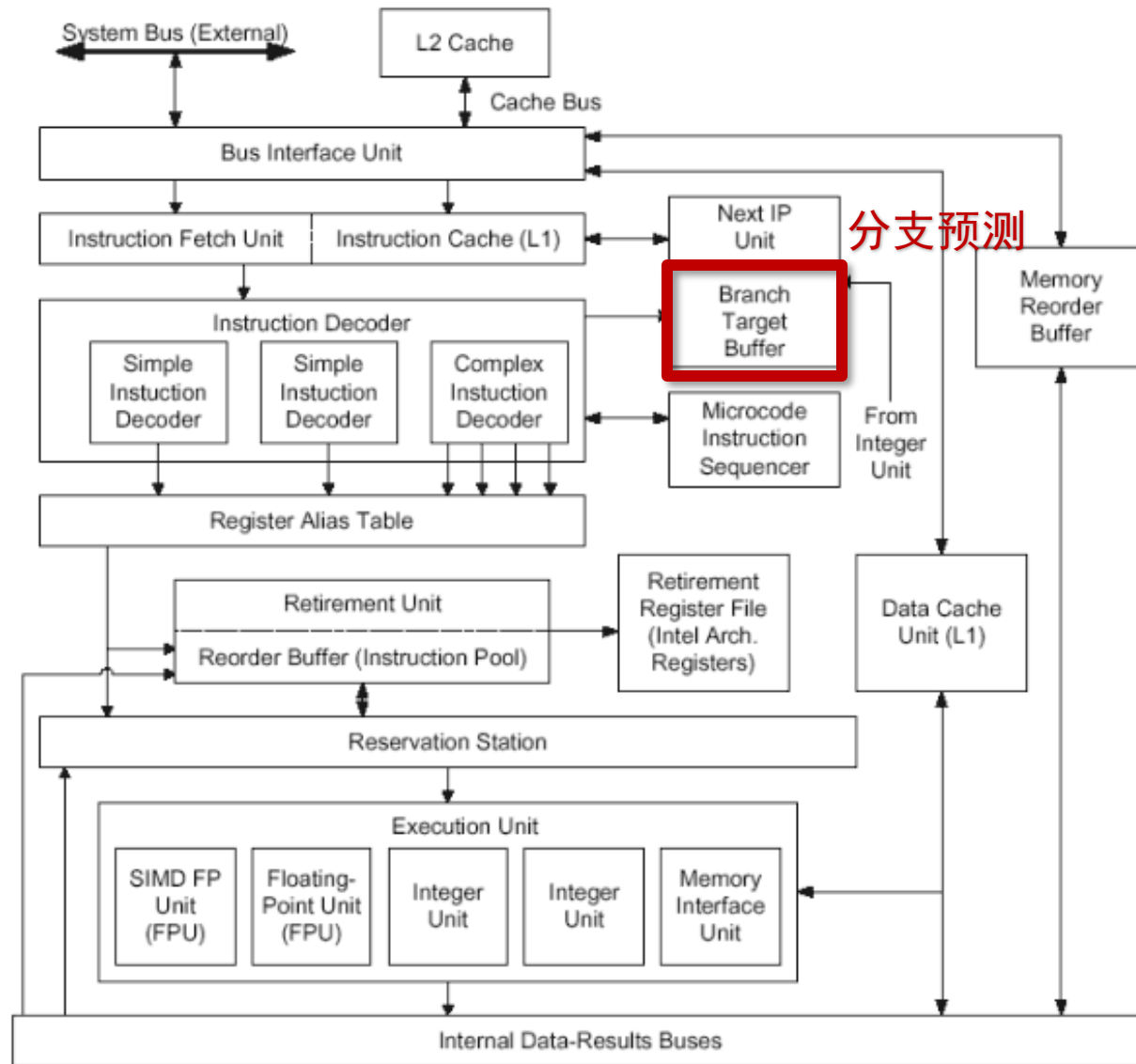
■ 分支预测

- 通过猜测分支中哪种情况最可能发生，然后推测执行这路分支的指令
- 一旦猜测错误，已经推测执行的指令结果全部放弃，重新执行正确分支
 - 代价巨大

■ 处理器的分支预测

- 硬件：基于自动机预测逻辑
- 软件：编译器协助分支预测

Intel Pentium 4



分支预测

加速技术示例

加速技术应用示例

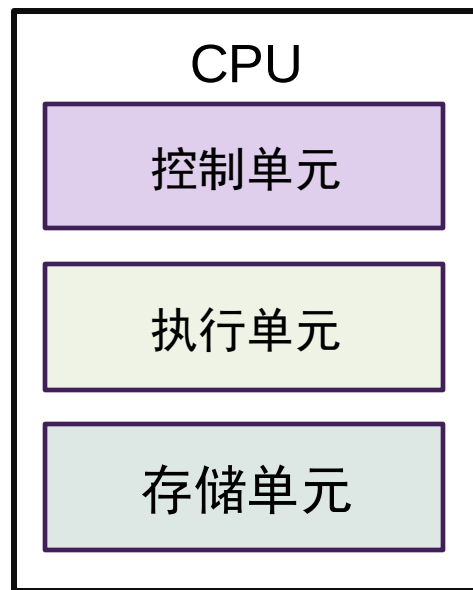
- 对 N 个浮点数，利用泰勒展开计算 $\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$

```
void sinx(int terms, float* x,
          int N, float* result)
{
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float number = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        result[i] = value;
    }
}
```

采用**流水线**的处理器
(一个周期完成**一条**指令)



加速技术应用示例

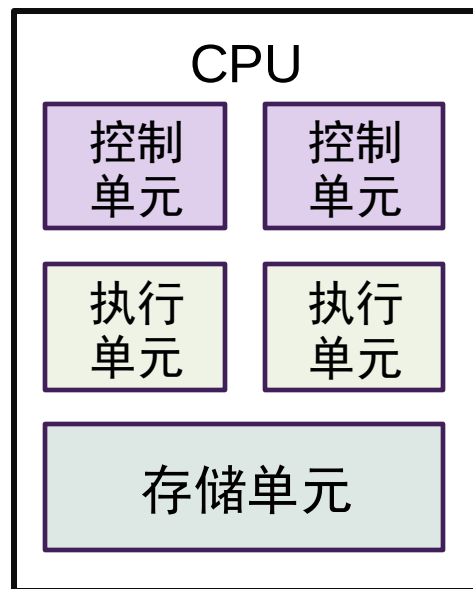
- 对 N 个浮点数，利用泰勒展开计算 $\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$

```
void sinx(int terms, float* x,
          int N, float* result)
{
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float number = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }
        result[i] = value;
    }
}
```

无数据依赖，相互独立

采用流水线+超标量的处理器
(一个周期至多完成两条指令)



加速技术应用示例

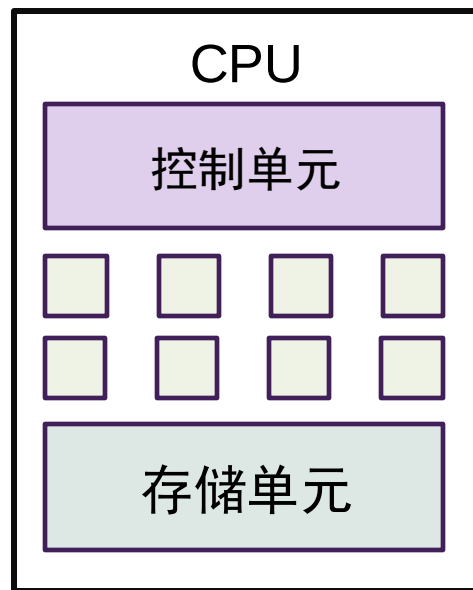
- 对 N 个浮点数，利用泰勒展开计算 $\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$

```
void sinx(int terms, float* x,
          int N, float* result)
{
    // 声明独立循环、循环展开
    forall (int i from 0 to N - 1)
    {
        float value = x[i];
        float number = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        result[i] = value;
    }
}
```

采用流水线+SIMD的处理器
(一个周期可完成一个 8-wide SIMD指令)



加速技术应用示例

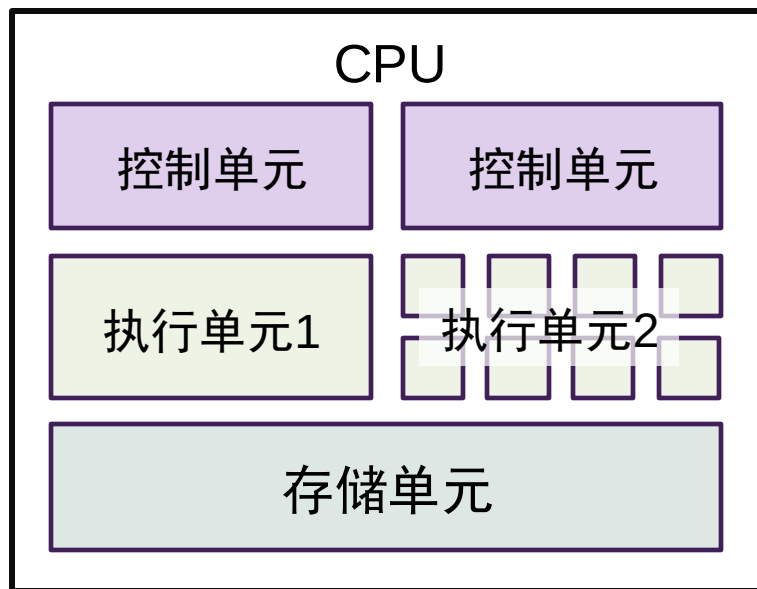
- 对 N 个浮点数，利用泰勒展开计算 $\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$

```
void sinx(int terms, float* x,
          int N, float* result)
{
    // 声明独立循环、循环展开
    forall (int i from 0 to N - 1)
    {
        float value = x[i];
        float number = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }
        result[i] = value;
    }
}
```

无数据依赖，相互独立

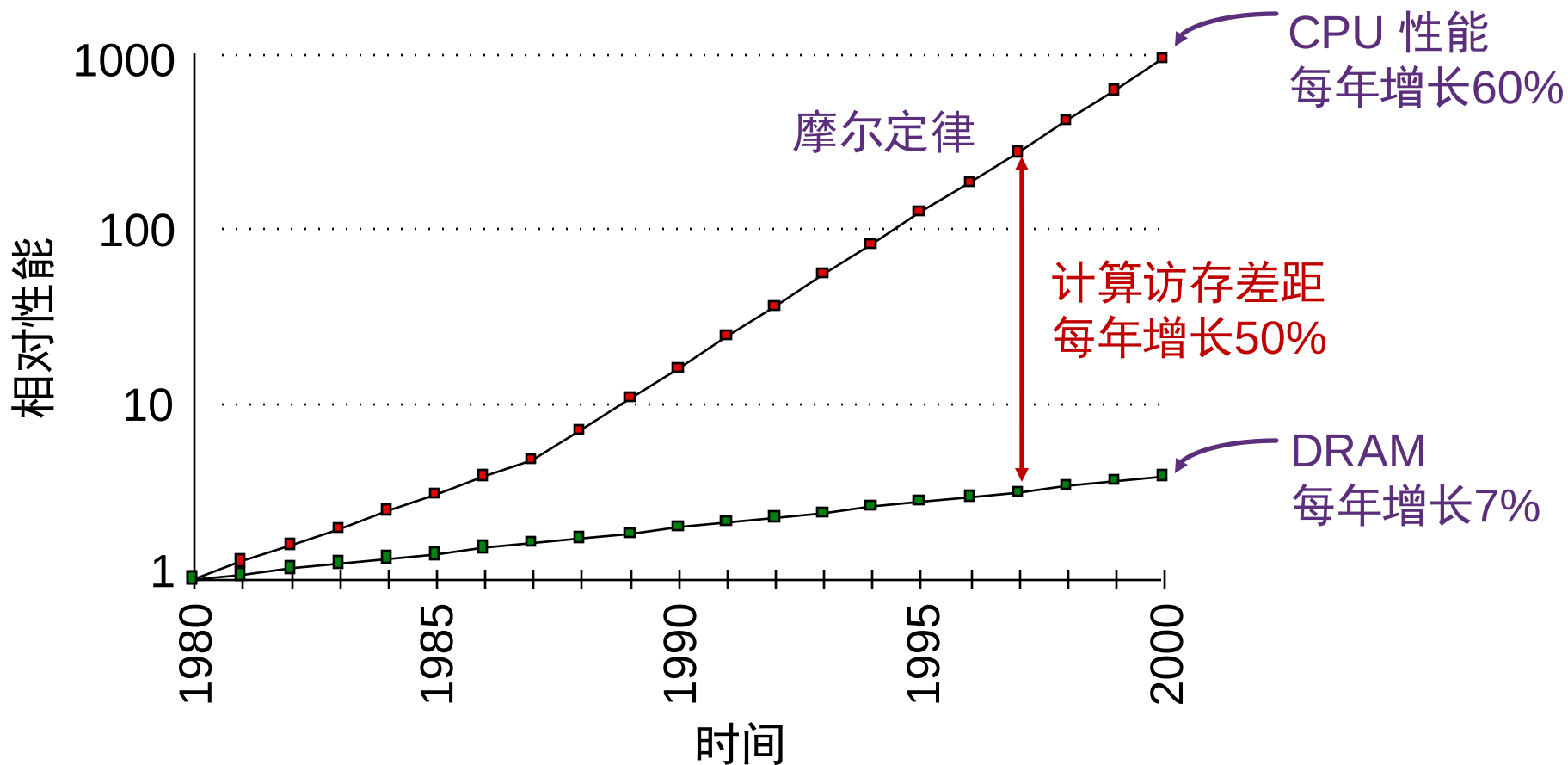
采用流水线+超标量+SIMD的处理器
(一个周期至多完成两条指令，
包含：一个标量指令
和一个 8-wide SIMD指令)



存储层次

计算访存性能差距（延迟）

- 计算访存性能差距越来越大



程序局部性

- 很多程序在访存上都具有两种局部性 (Locality)
 - 时间局部性 *temporal locality* : 最近访问的数据, 被再次访问
 - 空间局部性 *spatial locality*: 最近访问的数据, 其周边地址数据被访问

```
int sum[N]
int num[N][M]

for i in 0..N:
    for j in 0..M:
        sum[i] += num[i][j]
```

程序局部性

- 很多程序在访存上都具有两种局部性 (Locality)
 - 时间局部性 temporal locality : 最近访问的数据, 被再次访问
 - 空间局部性 spatial locality : 最近访问的数据, 其周边地址数据被访问

```
int sum[N]
int num[N][M]

for i in 0..N:
    for j in 0..M:
        sum[i] += num[i][j]
```

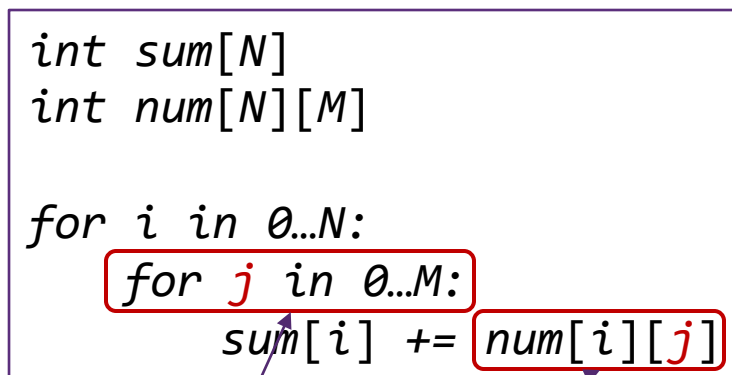
时间局部性: 内层循环反复使用 `sum[i]`

程序局部性

- 很多程序在访存上都具有两种局部性 (Locality)
 - 时间局部性 temporal locality : 最近访问的数据, 被再次访问
 - 空间局部性 spatial locality : 最近访问的数据, 其周边地址数据被访问

```
int sum[N]
int num[N][M]

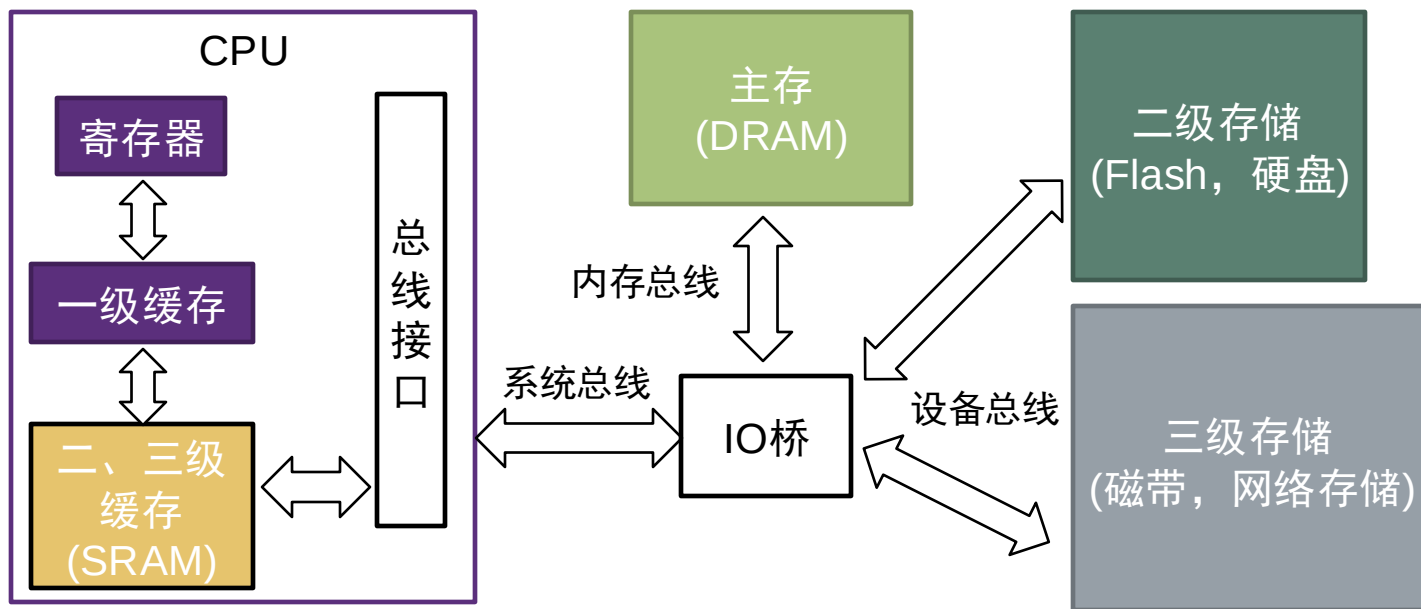
for i in 0..N:
    for j in 0..M:
        sum[i] += num[i][j]
```



空间局部性: 内层循环顺序遍历 $num[i]$

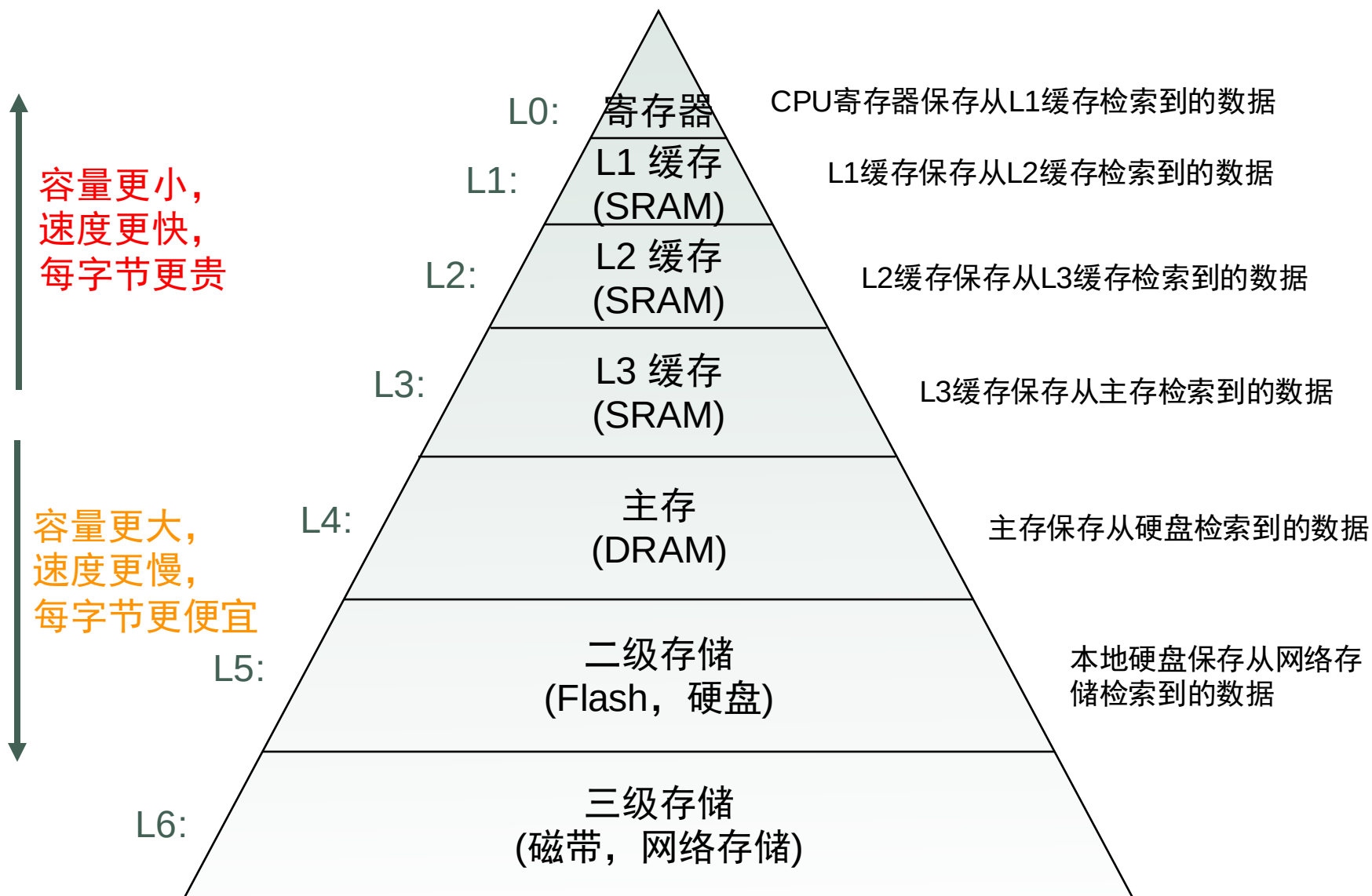
存储层次

- 存储层次试图适应并**利用程序局部性**，来提高程序执行性能



存储类型	寄存器、L1	L2/L3 Cache	主存	硬盘	磁带
速度	1ns	10ns	100ns	10ms	10s
大小	KB	MB	GB	TB	PB

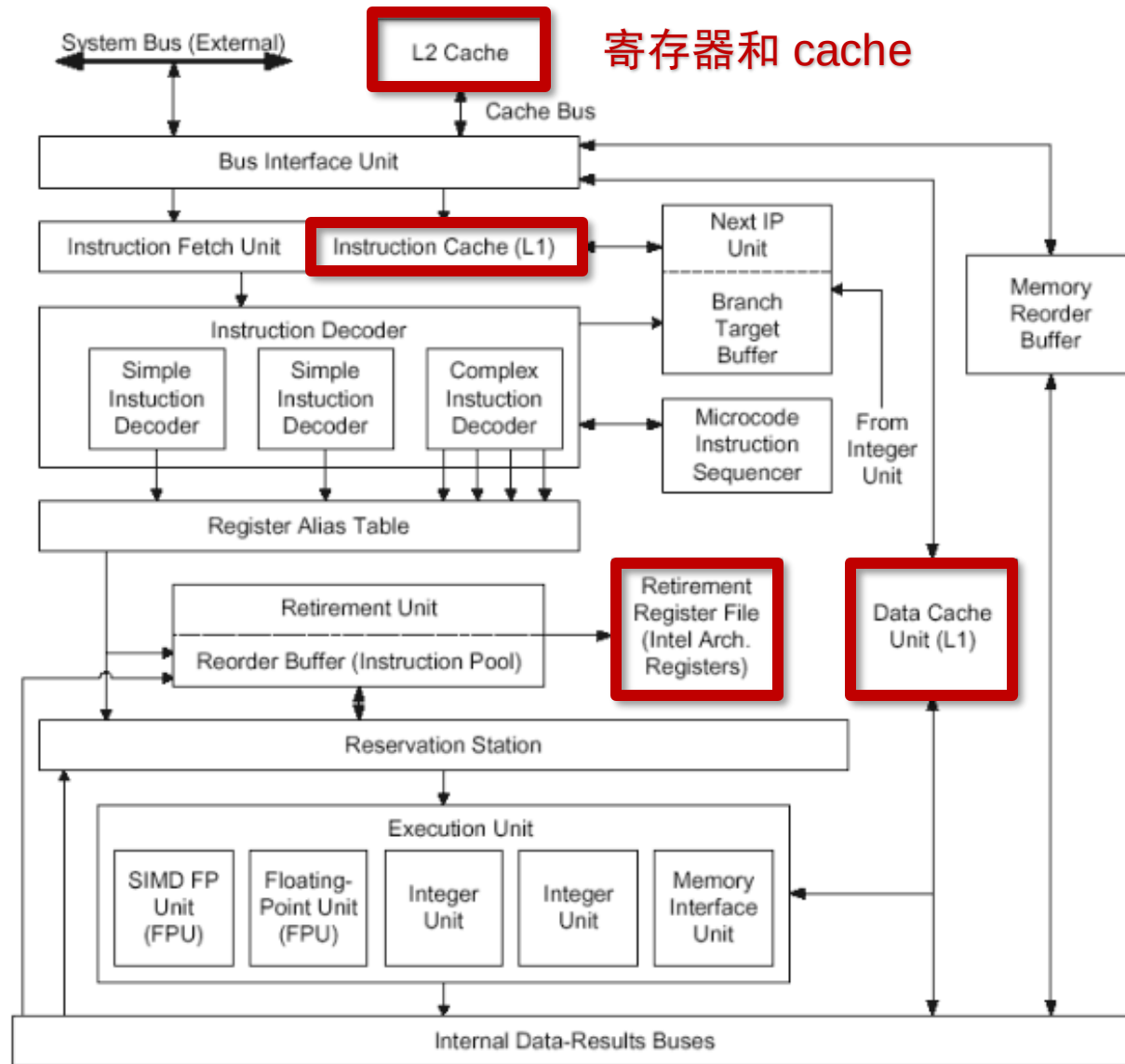
存储层次



优化访存性能 - 方法

1. 将数据保持在Cache中，通过数据重用来消除内存操作
 - 提高程序的时间局部性
2. 利用内存整块读取的高带宽特性，将一整块数据放在Cache中，访问该块内的数据可消除内存操作
 - 带宽每年增长23%（而延迟性能每年改善7%）
 - 提高程序的空间局部性
3. 通过并发内存访问充分利用内存带宽
 - 指令级并行、SIMD、多线程
4. 计算和访存重叠
 - 内存预取

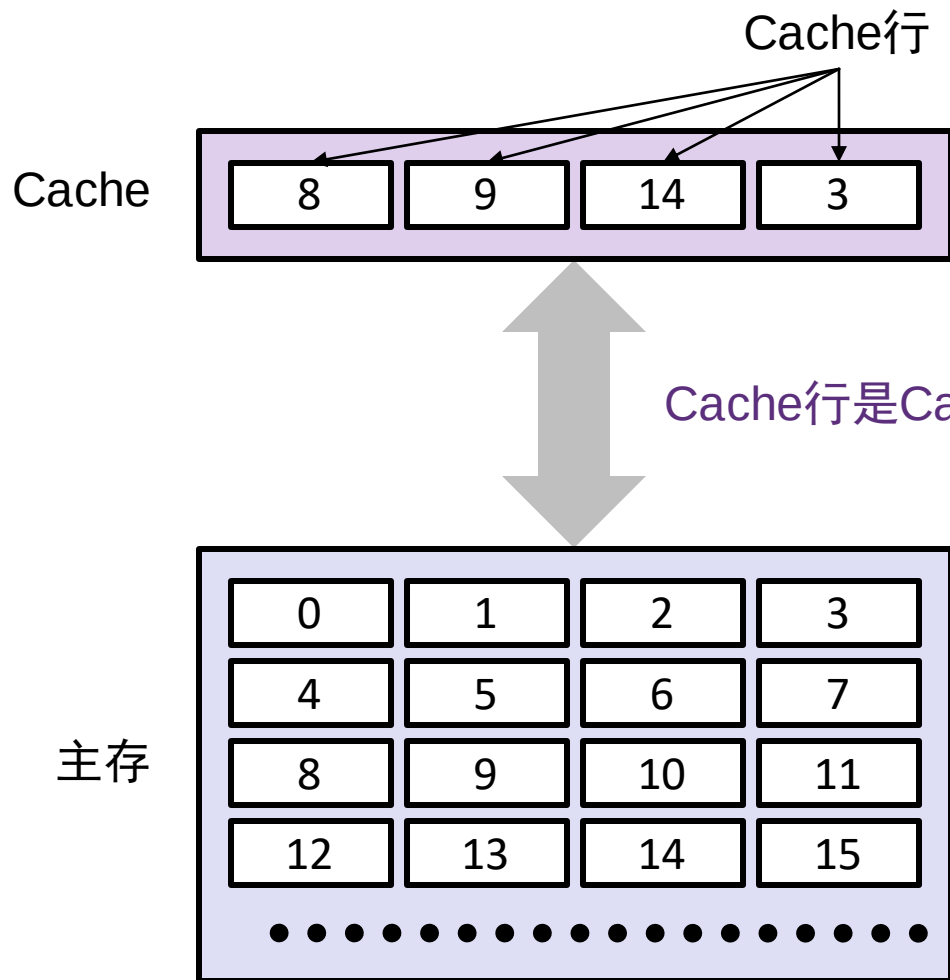
Intel Pentium 4



CACHE 基本概念

Cache 基本概念

- Cache是主存数据的**高速缓存**，其中保存的数据是**主存数据的拷贝**



Cache由Cache行（Cache Line）组成

简单起见，本例中Cache行大小为一个字节
典型Cache行大小为64字节

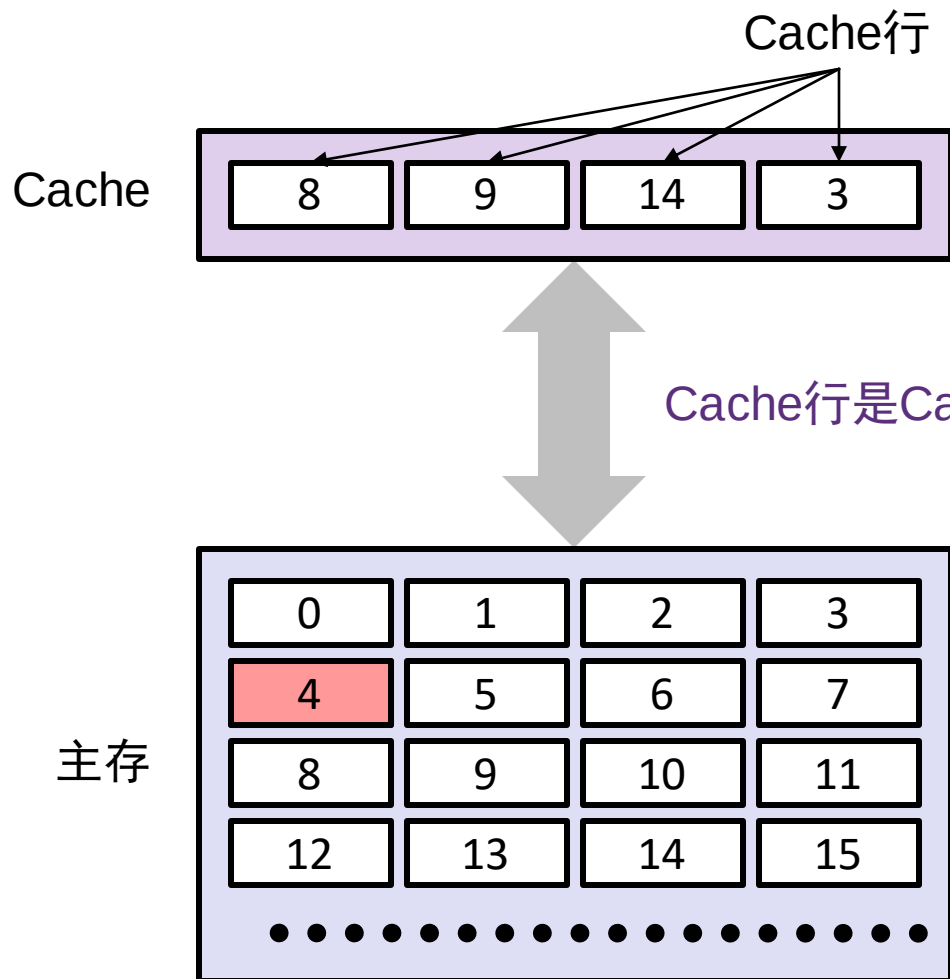
Cache行是Cache与主存进行数据交换的基本单位

注：访问Cache可访问Cache行的任何一个字节

主存可视作依据Cache行大小分块

Cache 基本概念

- Cache是主存数据的**高速缓存**，其中保存的数据是**主存数据的拷贝**



Cache由Cache行（Cache Line）组成

简单起见，本例中Cache行大小为一个字节
典型Cache行大小为64字节

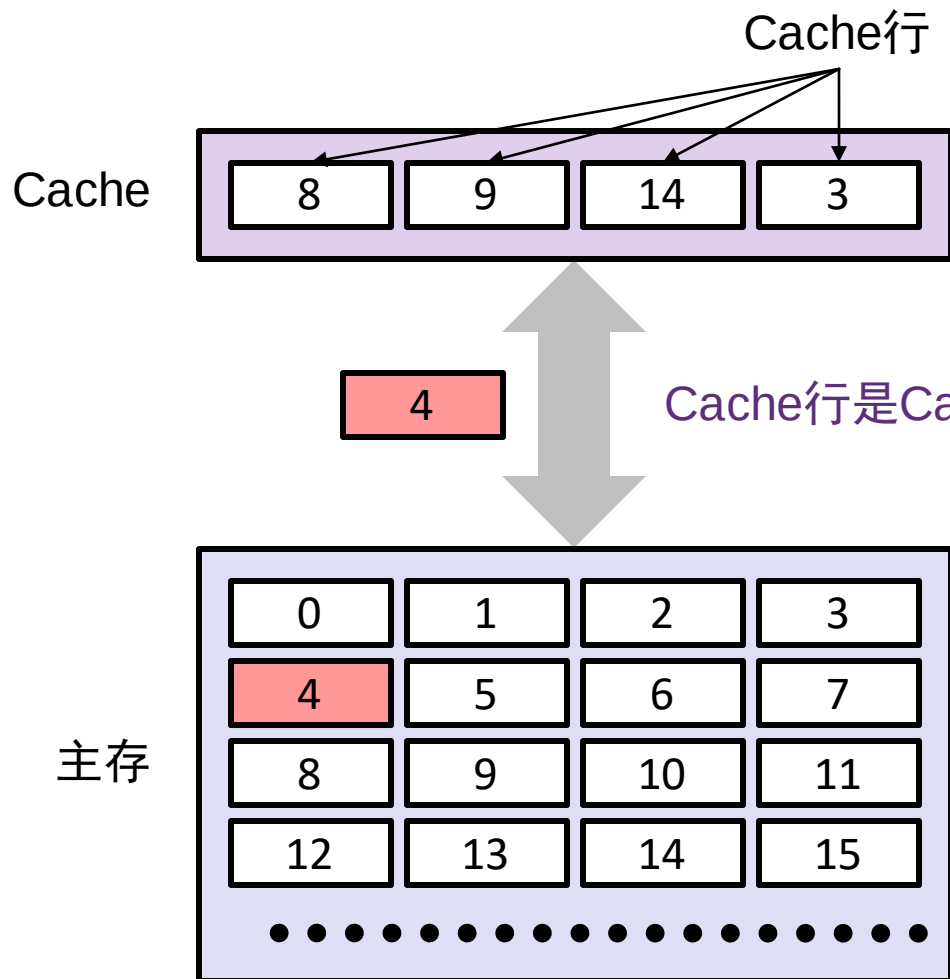
Cache行是Cache与主存进行数据交换的基本单位

注：访问Cache可访问Cache行的任何一个字节

主存可视作依据Cache行大小分块

Cache 基本概念

- Cache是主存数据的**高速缓存**，其中保存的数据是**主存数据的拷贝**



Cache由Cache行（Cache Line）组成

简单起见，本例中Cache行大小为一个字节
典型Cache行大小为64字节

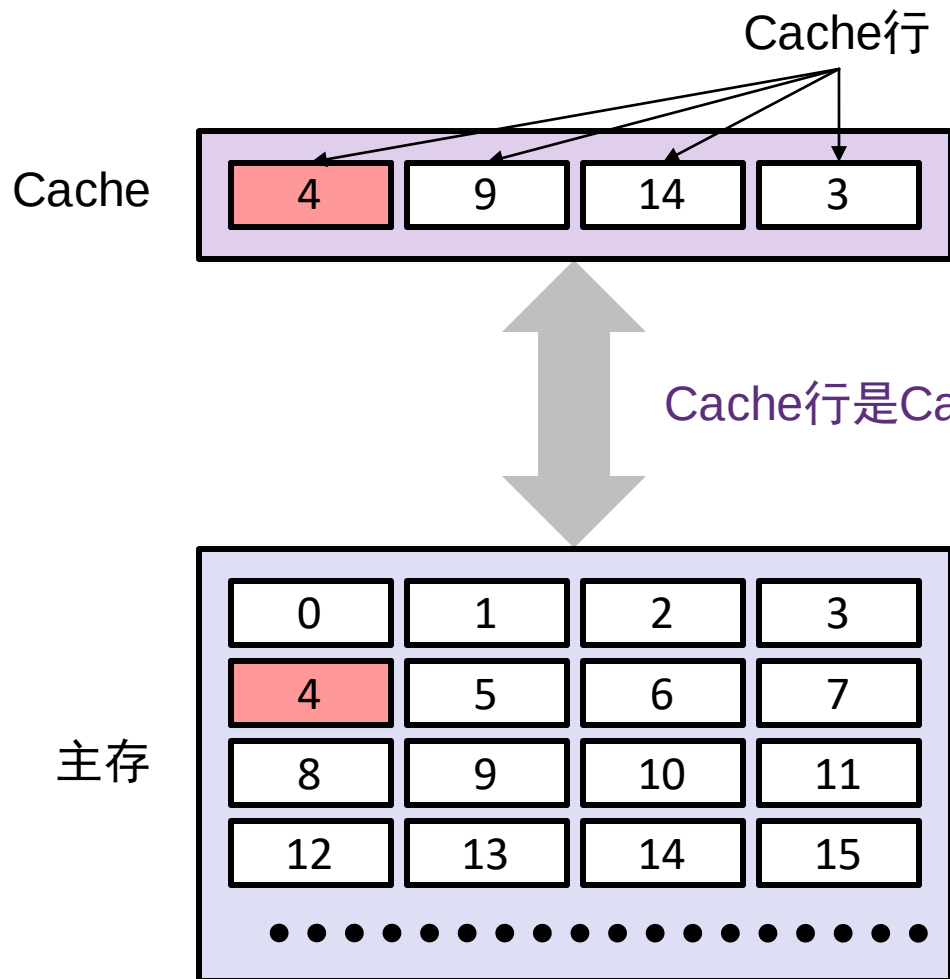
Cache行是Cache与主存进行数据交换的基本单位

注：访问Cache可访问Cache行的任何一个字节

主存可视作依据Cache行大小分块

Cache 基本概念

- Cache是主存数据的**高速缓存**，其中保存的数据是**主存数据的拷贝**



Cache由Cache行（Cache Line）组成

简单起见，本例中Cache行大小为一个字节
典型Cache行大小为64字节

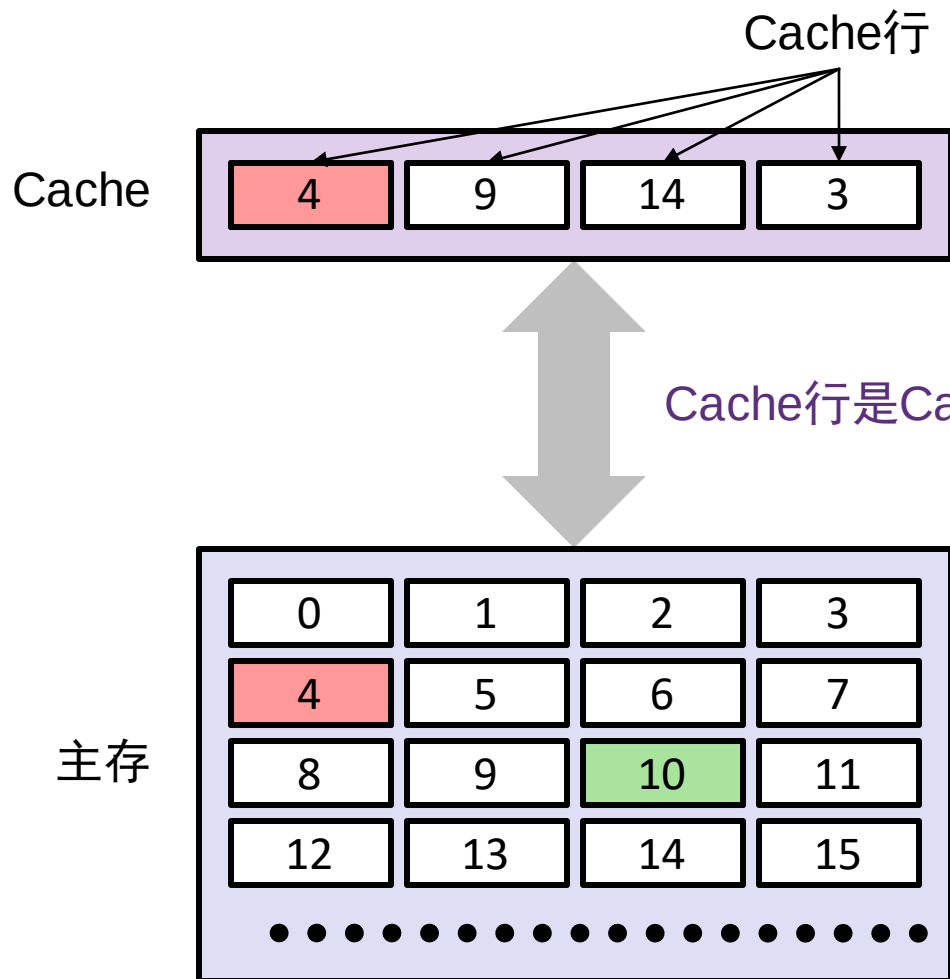
Cache行是Cache与主存进行数据交换的基本单位

注：访问Cache可访问Cache行的任何一个字节

主存可视作依据Cache行大小分块

Cache 基本概念

- Cache是主存数据的**高速缓存**，其中保存的数据是**主存数据的拷贝**



Cache由Cache行（Cache Line）组成

简单起见，本例中Cache行大小为一个字节
典型Cache行大小为64字节

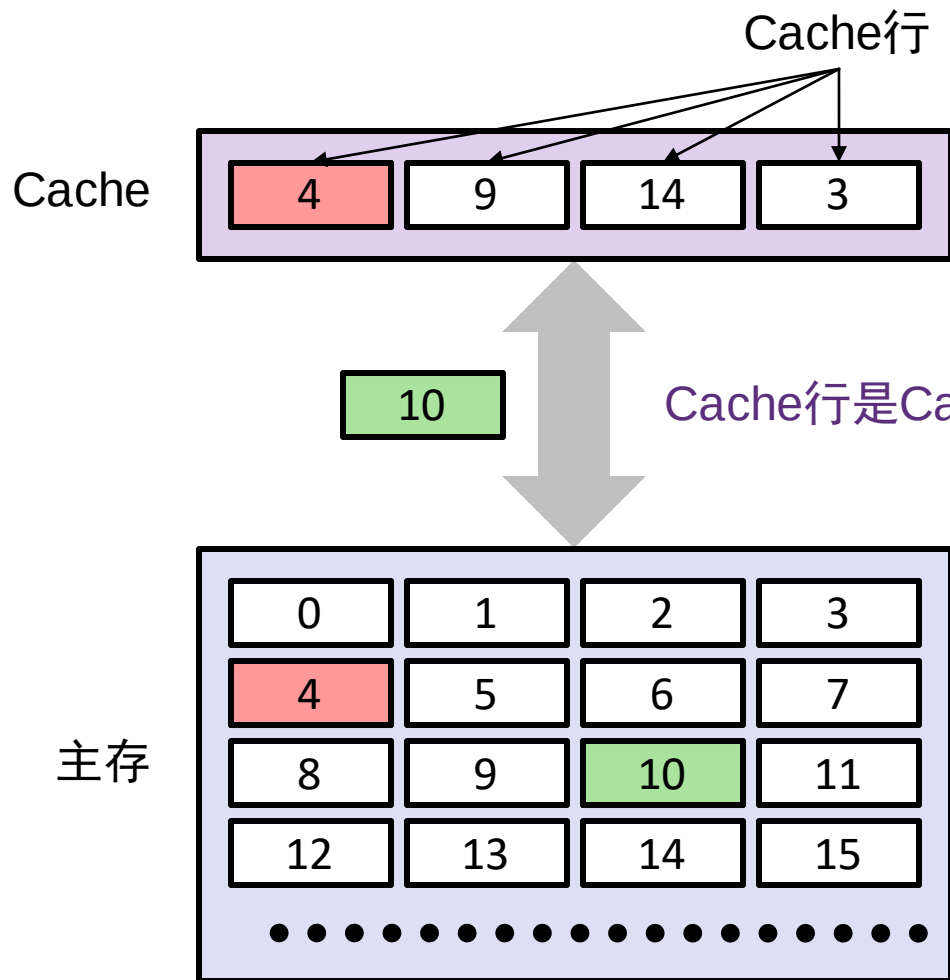
Cache行是Cache与主存进行数据交换的基本单位

注：访问Cache可访问Cache行的任何一个字节

主存可视作依据Cache行大小分块

Cache 基本概念

- Cache是主存数据的**高速缓存**，其中保存的数据是**主存数据的拷贝**



Cache由Cache行（Cache Line）组成

简单起见，本例中Cache行大小为一个字节
典型Cache行大小为64字节

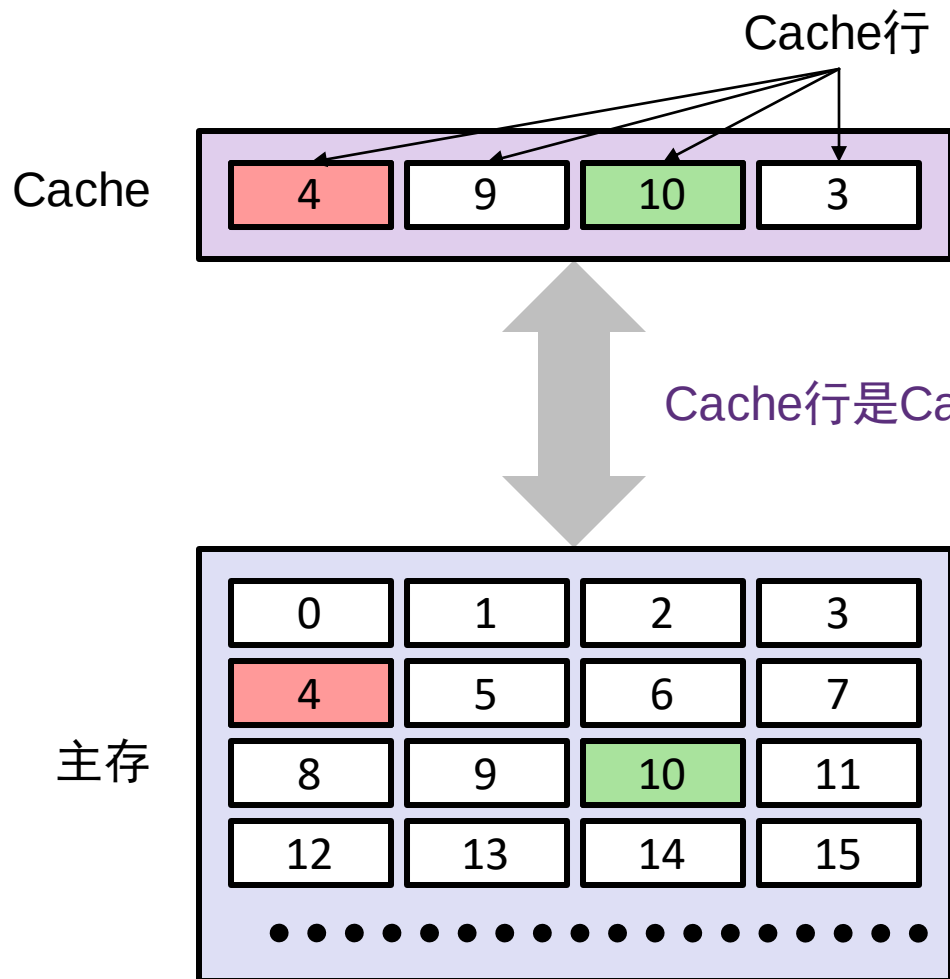
Cache行是Cache与主存进行数据交换的基本单位

注：访问Cache可访问Cache行的任何一个字节

主存可视作依据Cache行大小分块

Cache 基本概念

- Cache是主存数据的**高速缓存**，其中保存的数据是**主存数据的拷贝**



Cache由Cache行（Cache Line）组成

简单起见，本例中Cache行大小为一个字节
典型Cache行大小为64字节

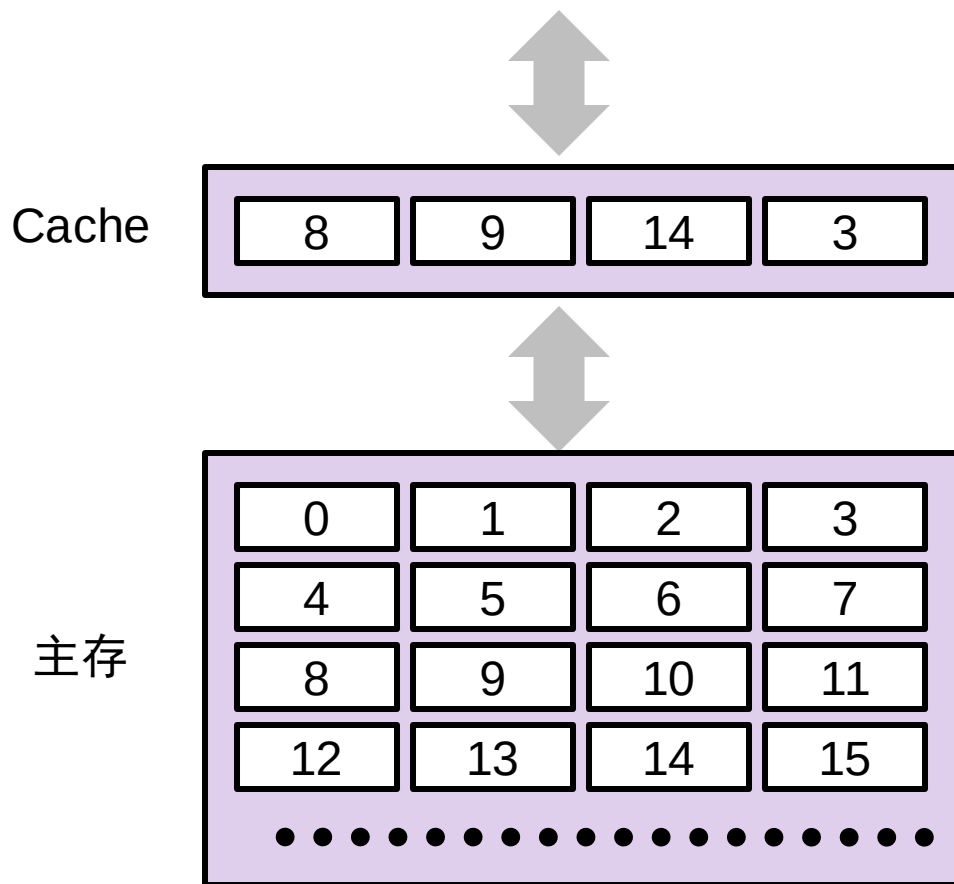
Cache行是Cache与主存进行数据交换的基本单位

注：访问Cache可访问Cache行的任何一个字节

主存可视作依据Cache行大小分块

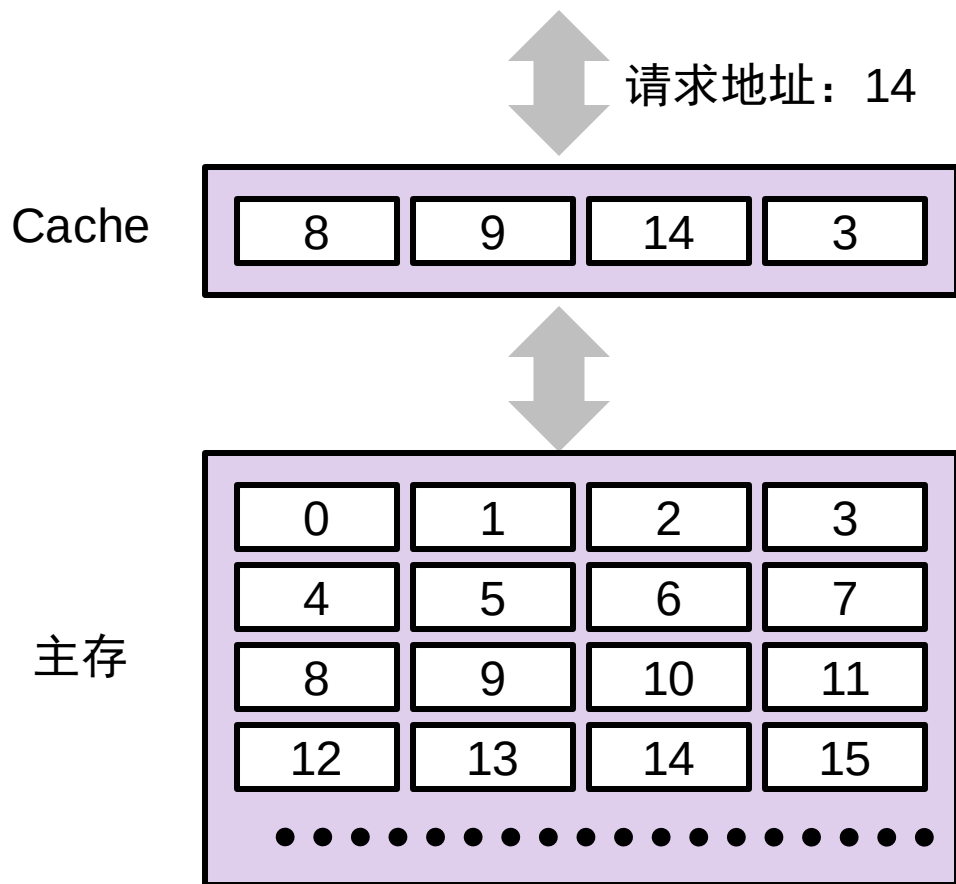
Cache 基本概念：访问命中

- 访问Cache的结果1：
 - 命中（Cache Hit）：待访问的数据已经存在于Cache中
 - 开销很小（<10周期）



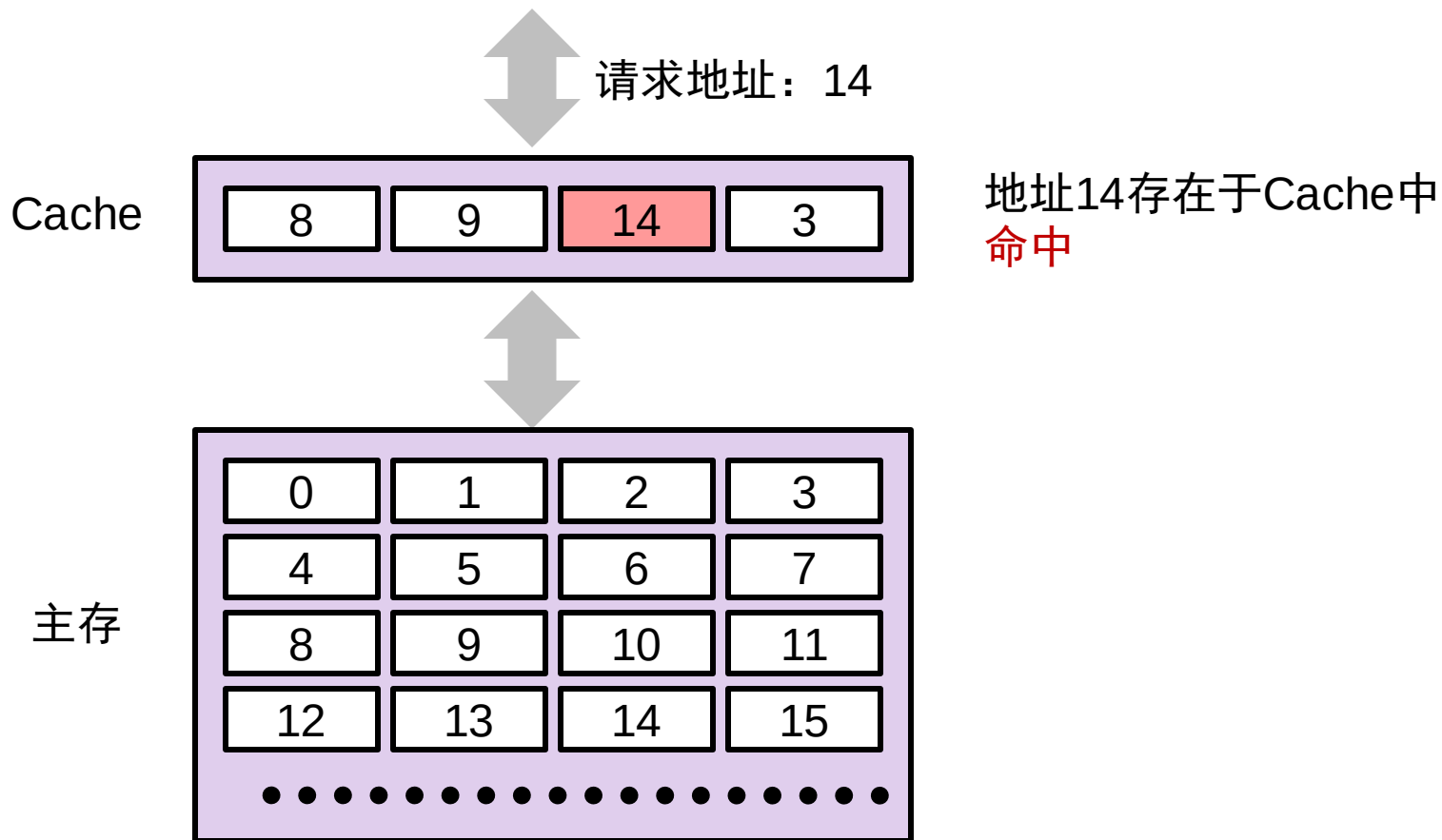
Cache 基本概念：访问命中

- 访问Cache的结果1：
 - 命中（Cache Hit）：待访问的数据已经存在于Cache中
 - 开销很小（<10周期）



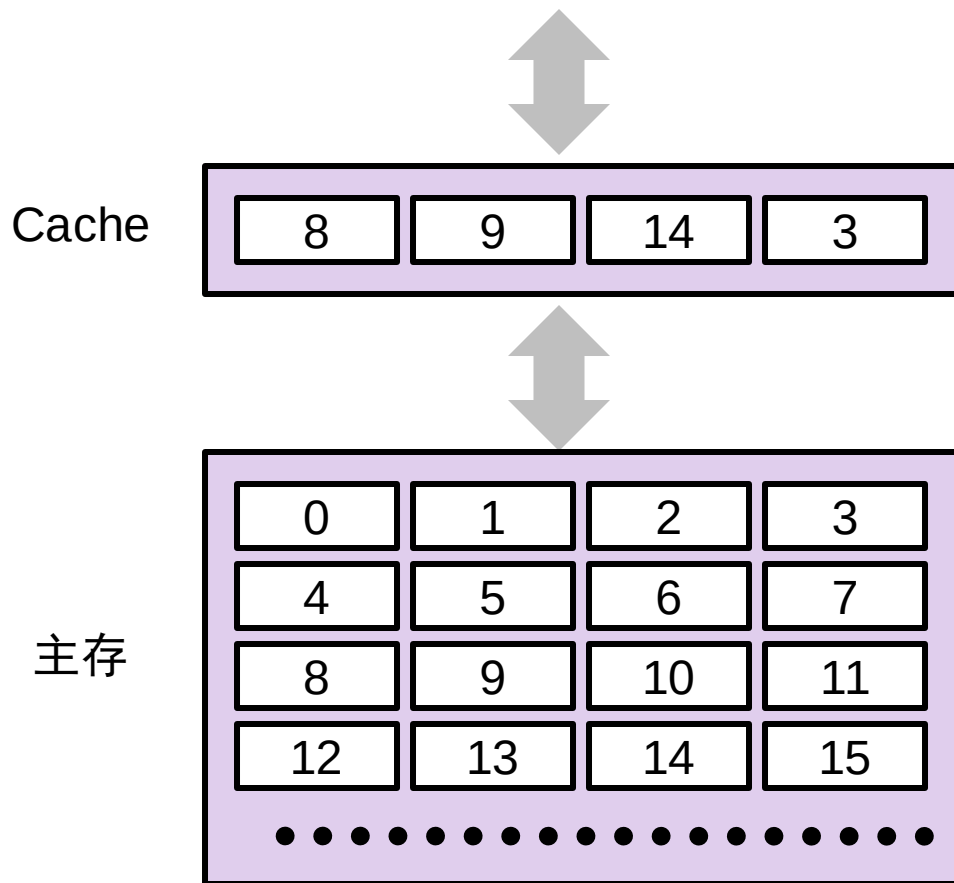
Cache 基本概念：访问命中

- 访问Cache的结果1：
 - 命中（Cache Hit）：待访问的数据已经存在于Cache中
 - 开销很小（<10周期）



Cache 基本概念：访问缺失

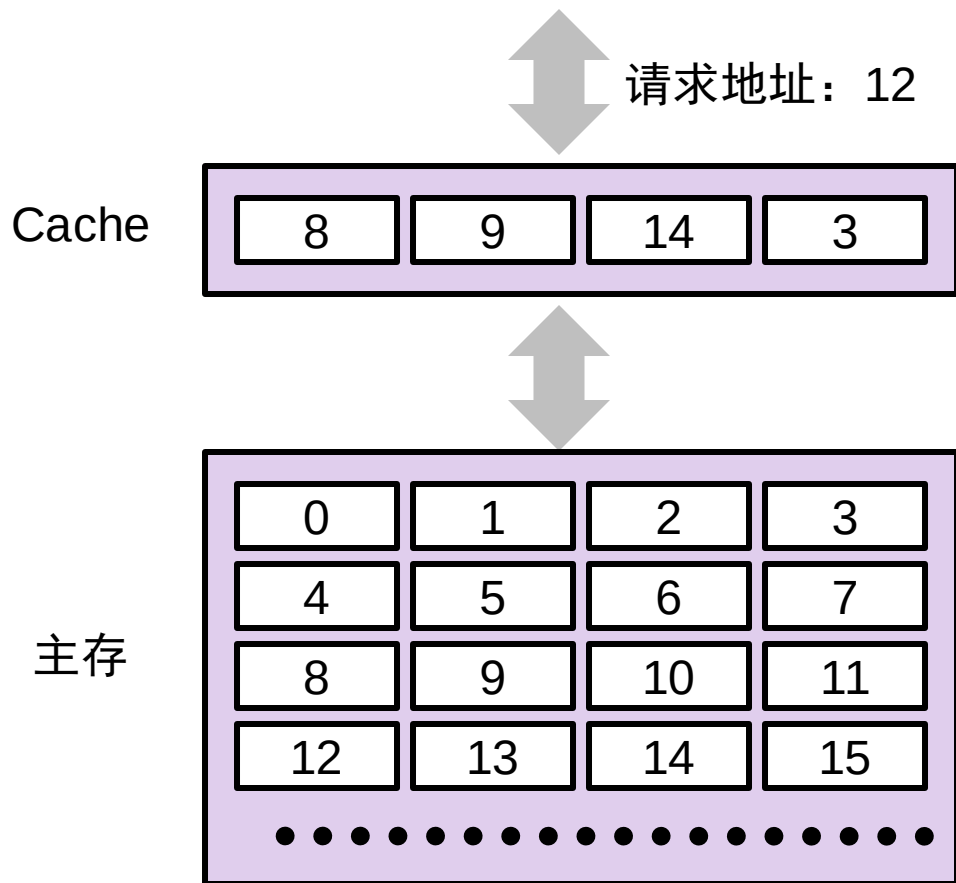
- 访问Cache的结果2：
 - **缺失（Cache Miss）**：待访问的数据不在Cache中，需要从主存中取得
 - **开销很大**（通常几百个周期）



Cache 基本概念：访问缺失

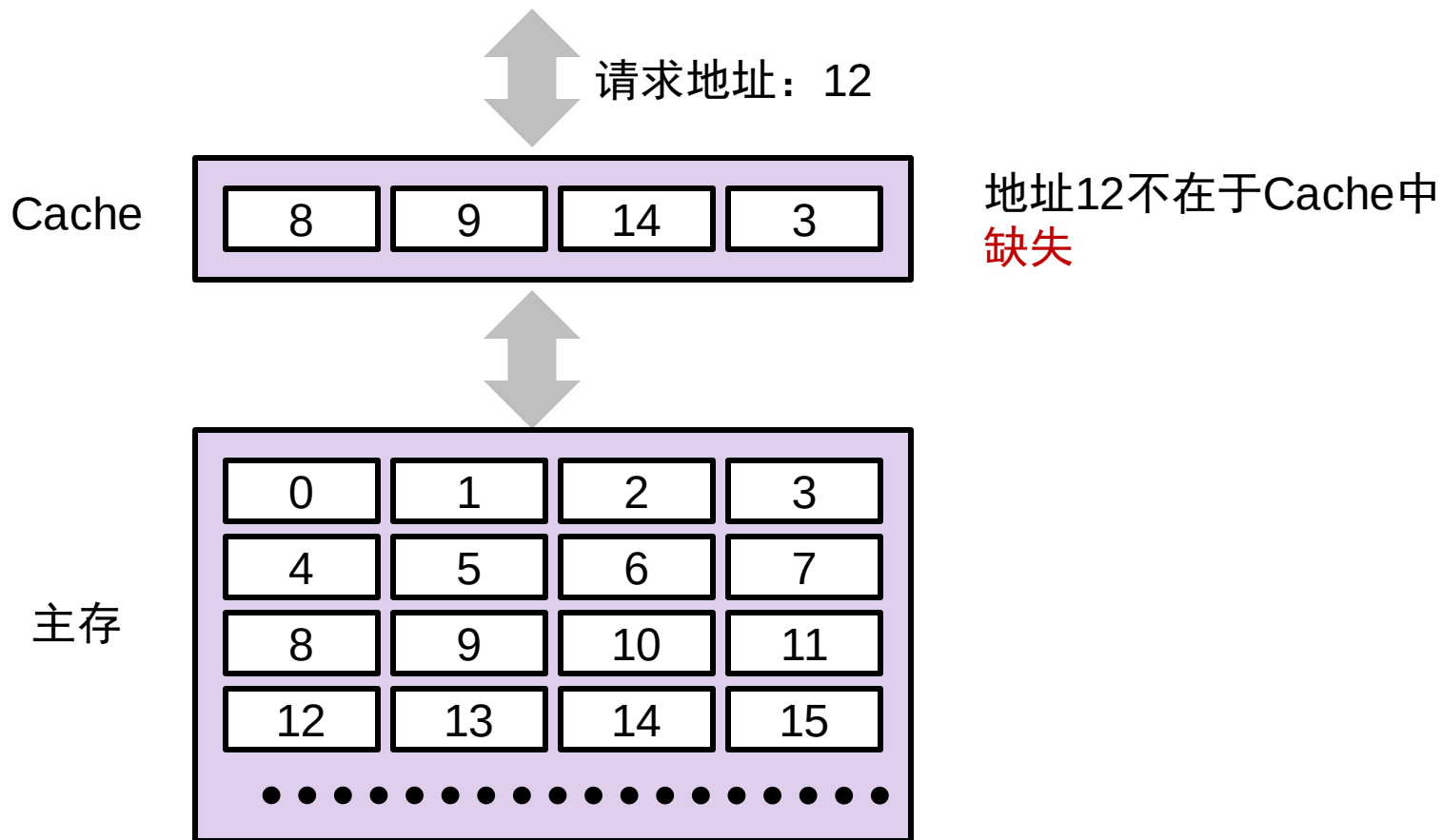
- 访问Cache的结果2:

- **缺失 (Cache Miss)**：待访问的数据不在Cache中，需要从主存中取得
 - **开销很大**（通常几百个周期）



Cache 基本概念：访问缺失

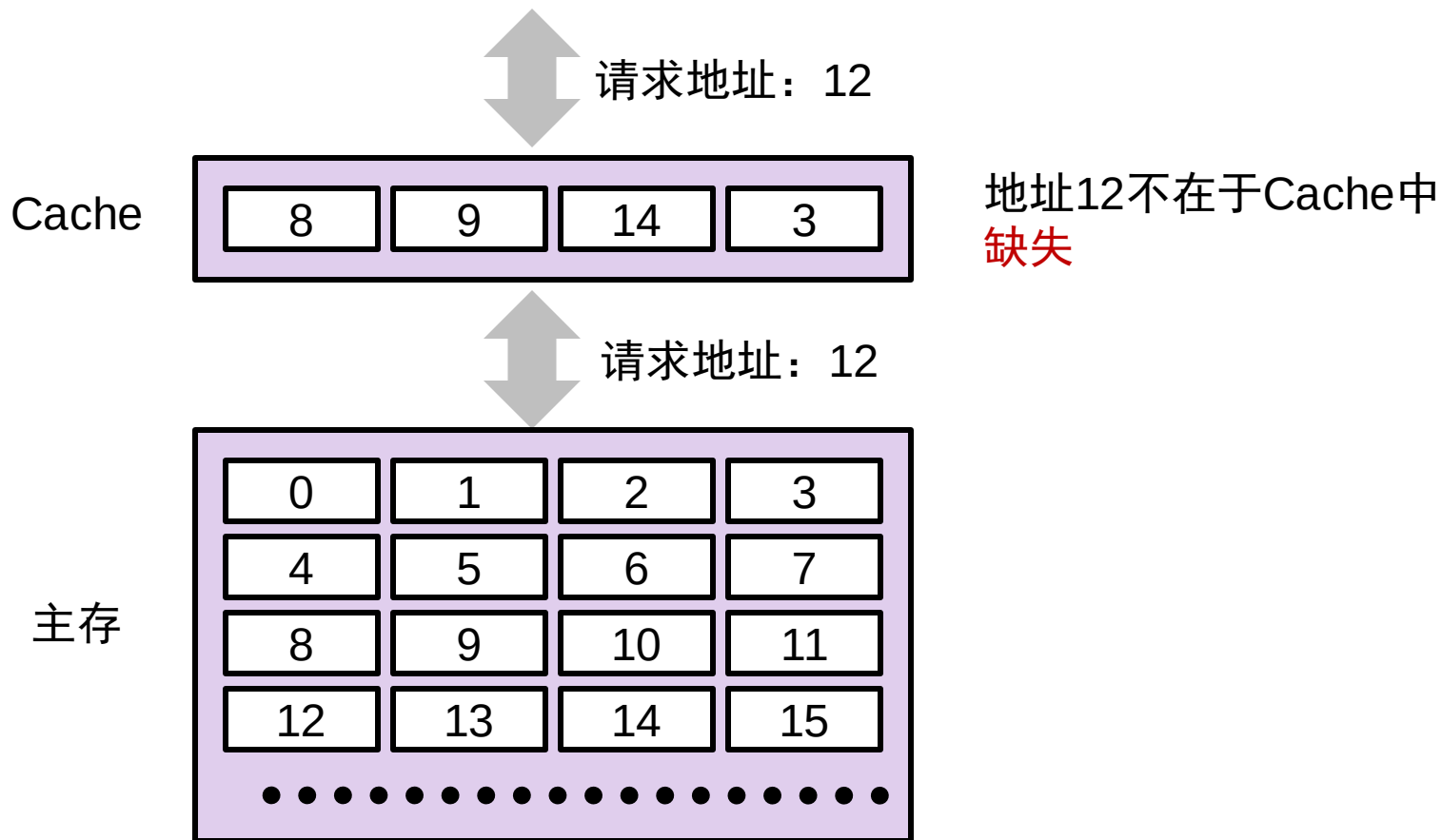
- 访问Cache的结果2：
 - **缺失（Cache Miss）**：待访问的数据不在Cache中，需要从主存中取得
 - 开销很大（通常几百个周期）



Cache 基本概念：访问缺失

- 访问Cache的结果2:

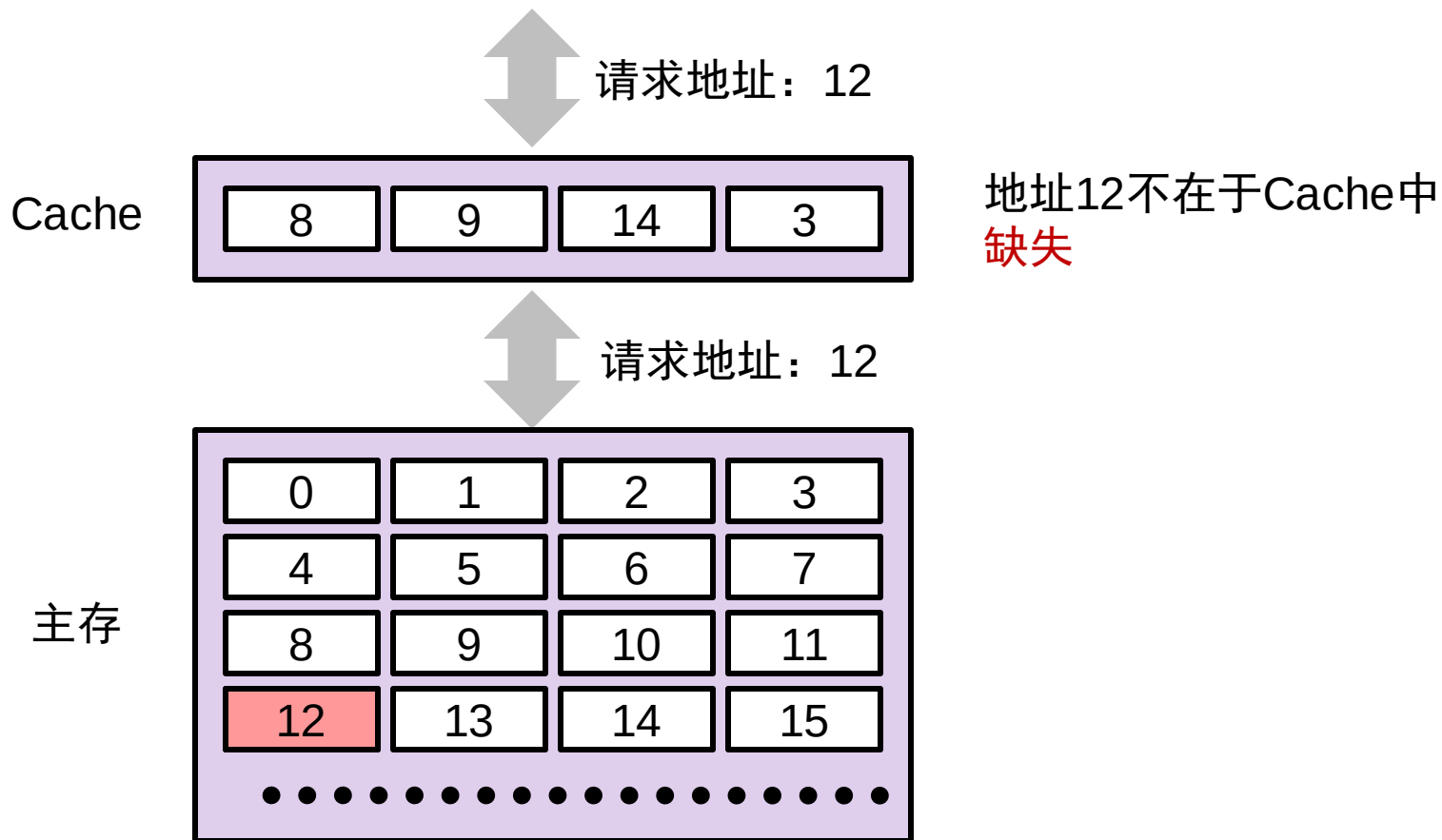
- **缺失 (Cache Miss)**：待访问的数据不在Cache中，需要从主存中取得
 - 开销很大（通常几百个周期）



Cache 基本概念：访问缺失

■ 访问Cache的结果2：

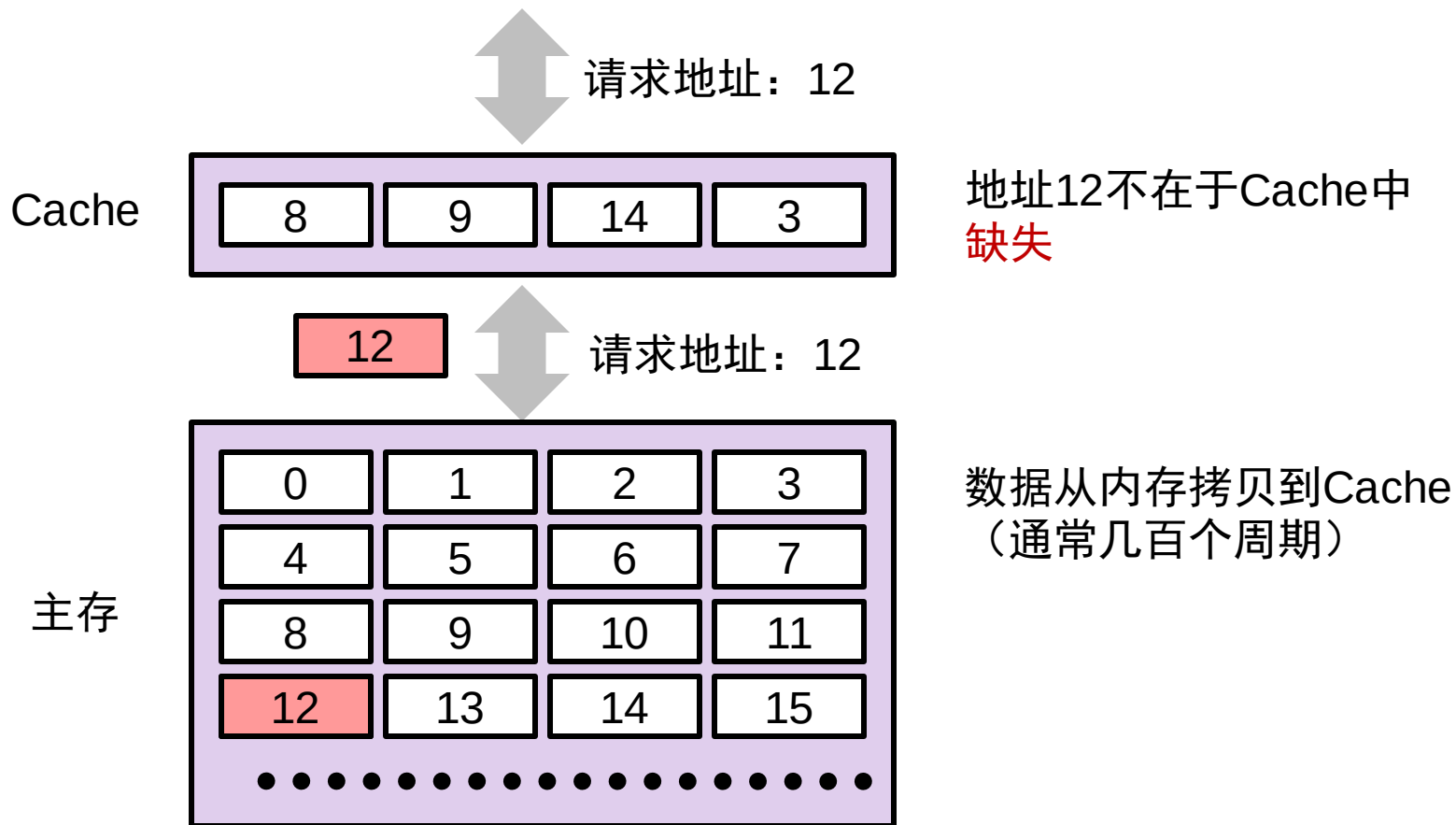
- **缺失（Cache Miss）**：待访问的数据不在Cache中，需要从主存中取得
 - 开销很大（通常几百个周期）



Cache 基本概念：访问缺失

■ 访问Cache的结果2：

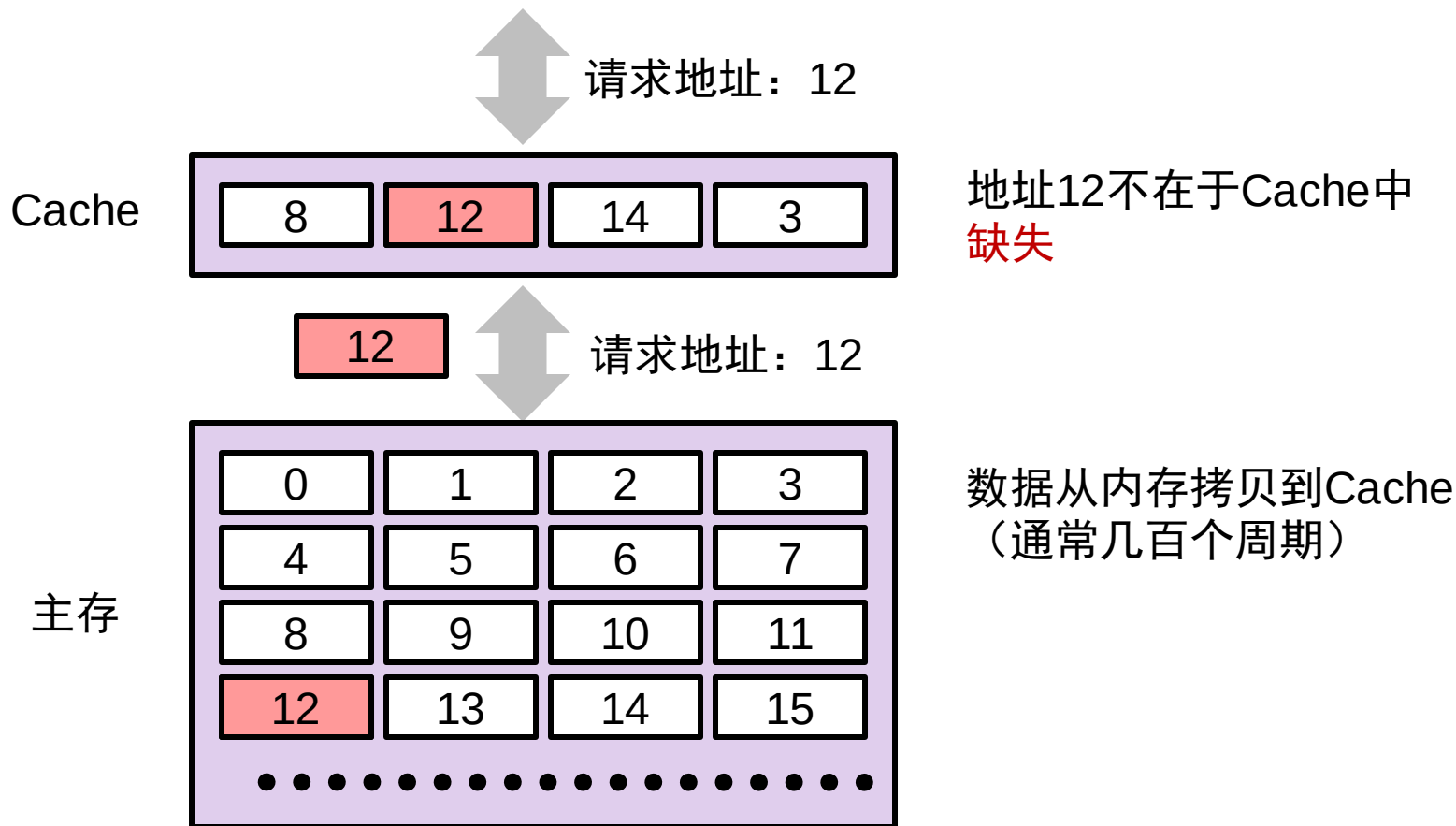
- **缺失（Cache Miss）**：待访问的数据不在Cache中，需要从主存中取得
 - 开销很大（通常几百个周期）



Cache 基本概念：访问缺失

■ 访问Cache的结果2：

- **缺失（Cache Miss）**：待访问的数据不在Cache中，需要从主存中取得
 - 开销很大（通常几百个周期）



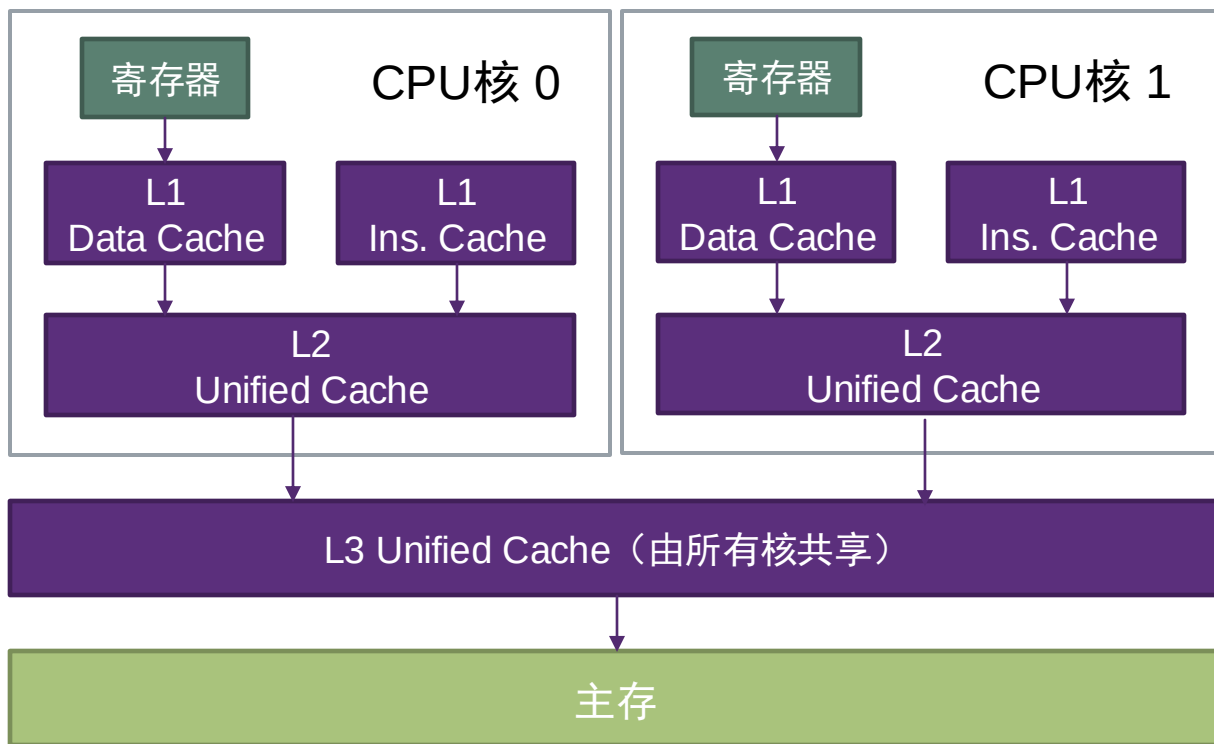
多级 Cache

■ 现代处理器的多级 Cache

- 例如：Intel Core i7
- L1 分数据、指令 Cache
 - 32 KB，访问：4周期
- L2 统一 Cache
 - 256 KB，访问：10周期
 - 每个核私有
- L3 共享 Cache
 - 8 MB，访问：40-75周期
 - 所有核共享
- Cache行大小：64字节

■ Cache对程序（包括操作系统）透明

- （通常情况下）由硬件管理
- 软件感知到的是访存性能差异



Cache 性能指标

- **Cache 缺失率** cache miss rate
 - 内存访问不在 cache 的比率
 - $\text{cache miss rate} = 1 - \text{cache hit rate}$
 - L1: 3-10% 典型值
 - L2: 相对更小一些、取决于 cache 大小（例如：~1%）
- 99% 命中率 vs. 97% 命中率**性能差别**
 - 假设cache命中需要1 cycle、cache缺失惩罚 100 cycles
 - **平均访问时间：**
 - 97% hits: $1 \text{ cycle} * 0.97 + 0.03 * 100 \text{ cycles} \approx 4 \text{ cycles}$
 - 99% hits: $1 \text{ cycle} * 0.99 + 0.01 * 100 \text{ cycles} \approx 2 \text{ cycles}$
 - **性能相差两倍** → 因此，cache miss rate 指标用的更多一些

如何避免访存开销：数据预取

■ 形式一：Cache Line 的整行数据交换

- 当访问 Cache 行的某个字节时，整个 Cache 行均会被取入，从而下次访问同行其它字节时 Cache 命中（64 Byte）

■ 形式二：基于硬件的自动预取（BIOS设置）

- 硬件识别程序的访问模式，在访问数据前预取数据
- 通常来说，可识别的访问模式是线性访问模式
 - 例如，顺序访问，带步长的顺序访问

■ 形式三：基于指令的软件预取

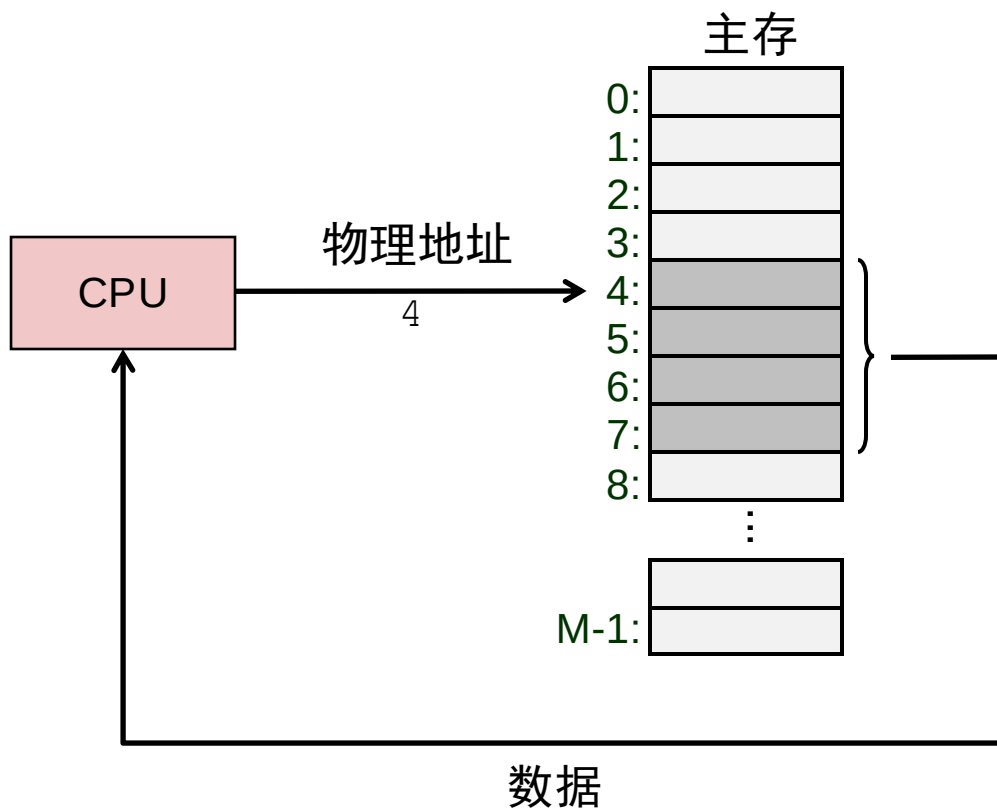
- T0：存放在所有级别的 Cache 中
- T1：存放在除L1外的 Cache 中
- T2：存放在除 L1、L2 外的 Cache 中
- NTA：同 T0，但在使用1次后可丢弃

```
#include <xmmintrin.h>
enum _mm_hint {
    _MM_HINT_T0 = 3,
    _MM_HINT_T1 = 2,
    _MM_HINT_T2 = 1,
    _MM_HINT_NTA = 0
};
void _mm_prefetch(void *p, int i);
// 预取函数
```

虚存

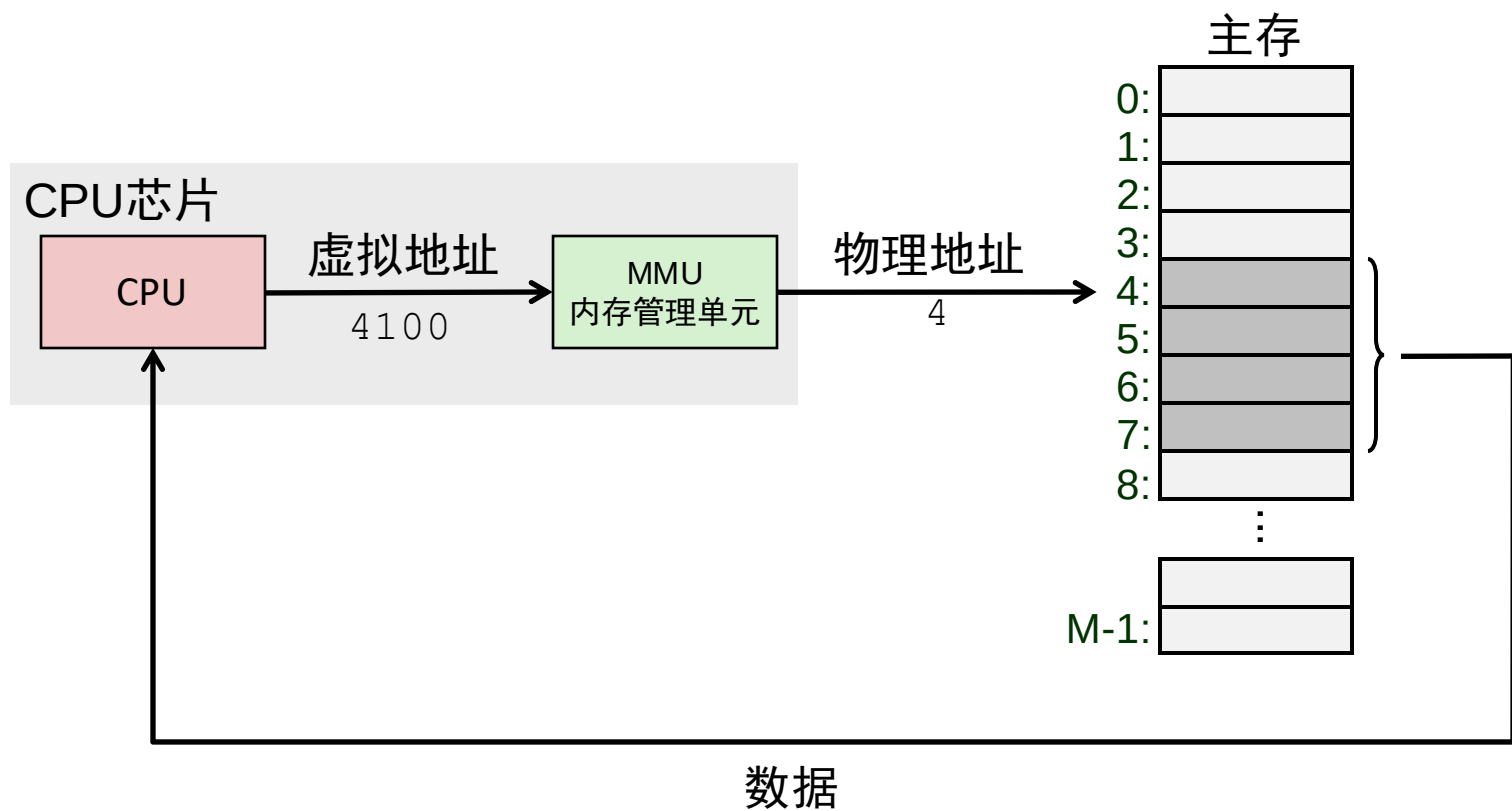
物理地址

- 使用物理地址寻址的系统
 - 用于“简单”系统，如电梯、数码相机等设备中的嵌入式微控制器



虚拟地址

- 虚存：使用**虚拟地址**寻址的系统
 - 用于所有现代服务器、笔记本电脑等
 - 计算机系统的重要思想之一

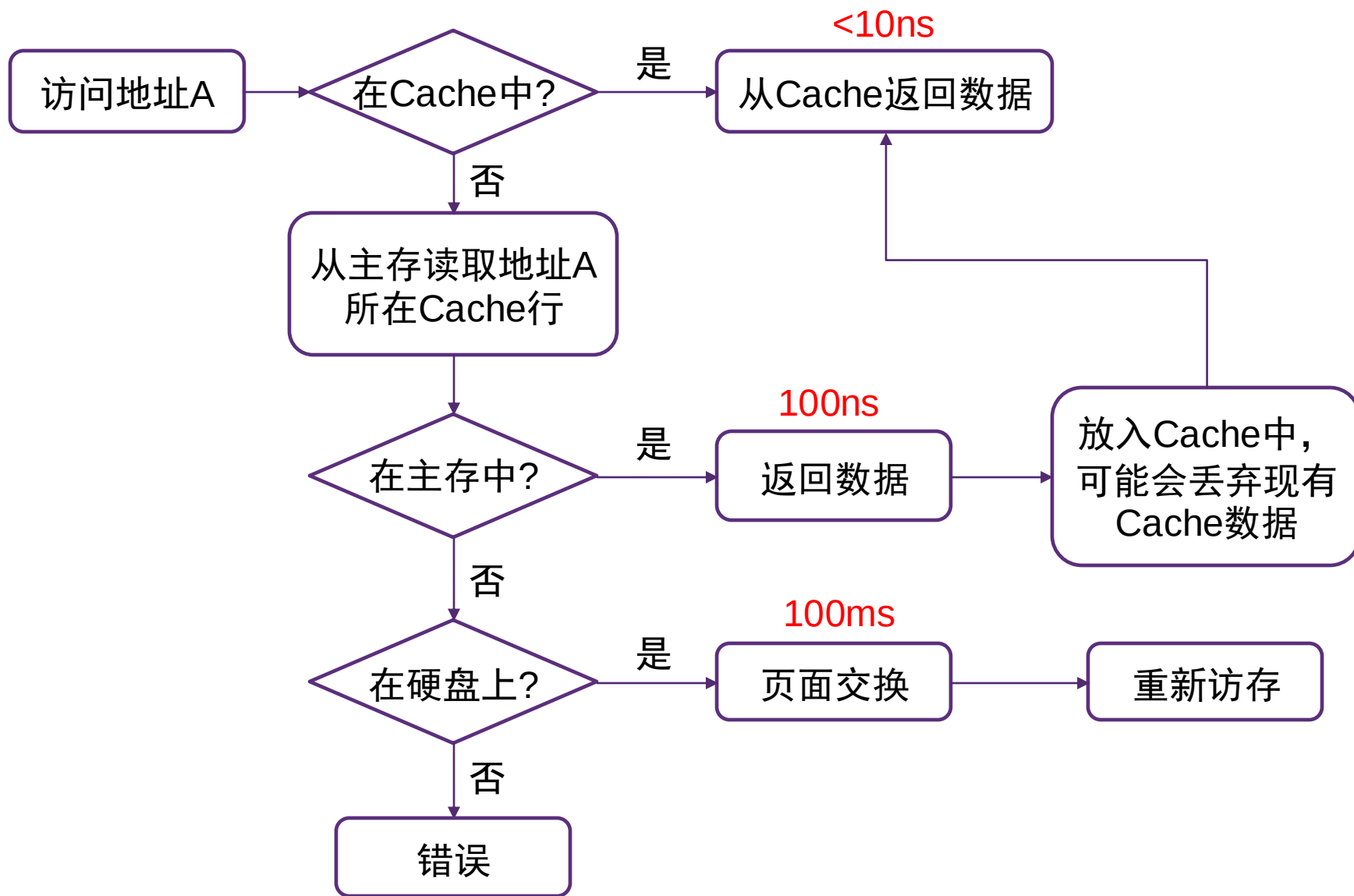


进程

- 进程概念：
 - 进程是一个运行程序的实例
 - 计算机系统中一个重要的概念
- 进程提供两个**关键抽象**：
 1. **逻辑控制流**
 - 每个程序似乎独有处理器的使用
 - 通过**上下文切换**实现
 2. **私有地址空间**
 - 每个程序似乎独有内存的使用
 - 通过**虚存机制**实现



基本访存流程



处理器峰值浮点性能

处理器峰值浮点性能

- Intel Xeon 8280 单精度峰值浮点性能: 2.4 TFlops
- Intel Xeon 6142 单精度峰值浮点性能: 2.7 TFlops
- 如何计算处理器峰值浮点性能?

如何计算处理器浮点峰值性能

- 浮点性能 = 核数 × 单核主频 × 单核每周期浮点处理指令数
 - 核数与单核主频容易获取
- 例如, Intel Xeon Gold 6142

# of Cores	?	核数	16
Processor Base Frequency	?	基频	2.60 GHz
Max Turbo Frequency	?	峰值频率	3.70 GHz

如何计算处理器浮点峰值性能

- 浮点性能 = 核数 × 单核主频 × 单核每周期浮点处理指令数
 - 峰值性能与向量化指令相关

- 例如, Intel Xeon Gold 6142

支持 512bit 向量化指令
= 16 float
= 8 double

Instruction Set Extensions ?

Intel® SSE4.2, Intel® AVX, Intel® AVX2, Intel® AVX-512

of AVX-512 FMA Units ?

2

of AVX-512 FMA Units



Intel® Advanced Vector Extensions 512 (AVX-512), new instruction set extensions, delivering ultra-wide (512-bit) vector operations capabilities, with up to 2 FMAs (Fused Multiply Add instructions), to accelerate performance for your most demanding computational tasks.

每个AVX 512乘加单元
每周期最多2个乘加指令

单核每周期单精度浮点指令数 = 2 (FMA单元个数) × 2 (2个指令/单元) × 16 = 64

如何计算处理器浮点峰值性能

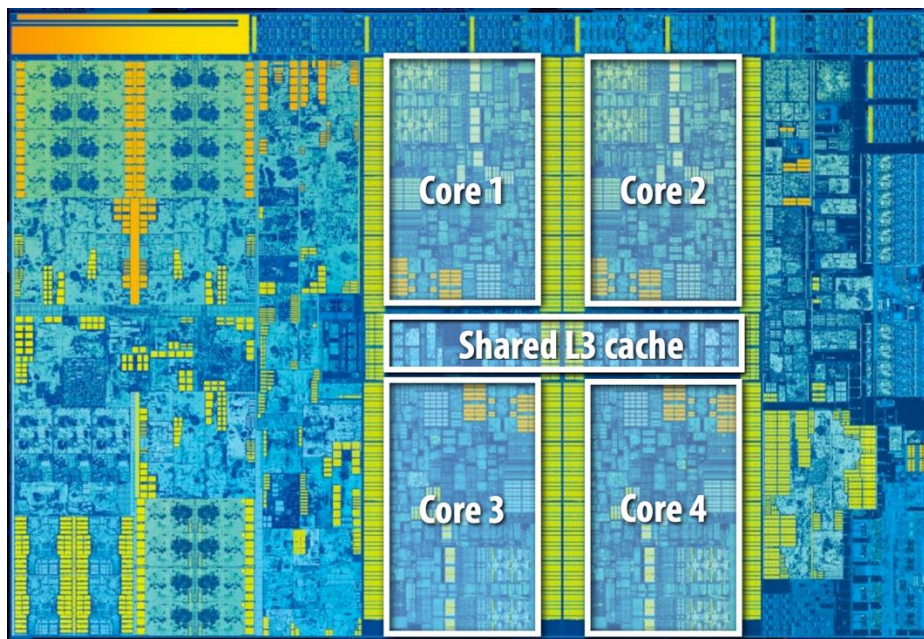
- 浮点性能 = 核数 × 单核主频 × 单核每周期浮点处理指令数
- 例如, Intel Xeon Gold 6142
 - 核数: 16
 - 单核基频: 2.6 GHz
 - 单核每周期单精度浮点指令数: 64
- **单精度**峰值浮点性能 = $16 \times 2.6 \times 64 = 2,662.4$ GFlops/s
- **双精度**峰值浮点性能 = $16 \times 2.6 \times 32 = 1,331.2$ GFlops/s

参考资料: <https://www.jianshu.com/p/b9d7126b08cc>
<https://colfaxresearch.com/xeon-2017/>

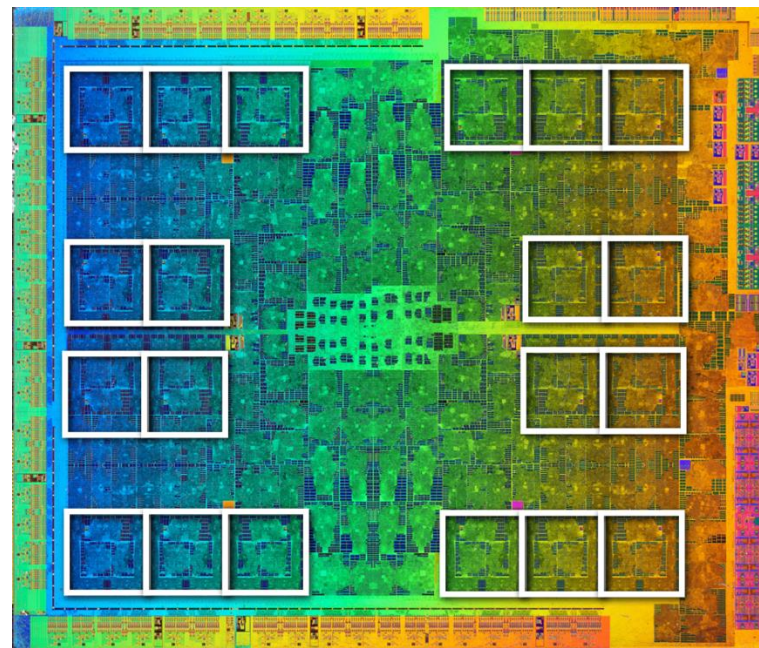
多核处理器

多核处理器

■ 多核处理器示例-1



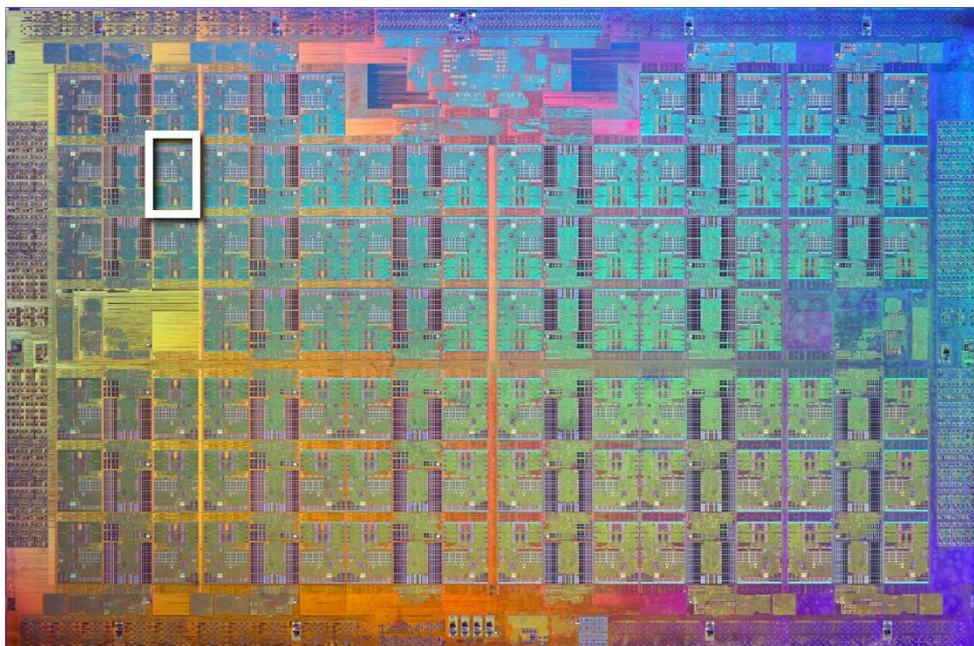
Intel “Skylake” Core i7 4 核处理器



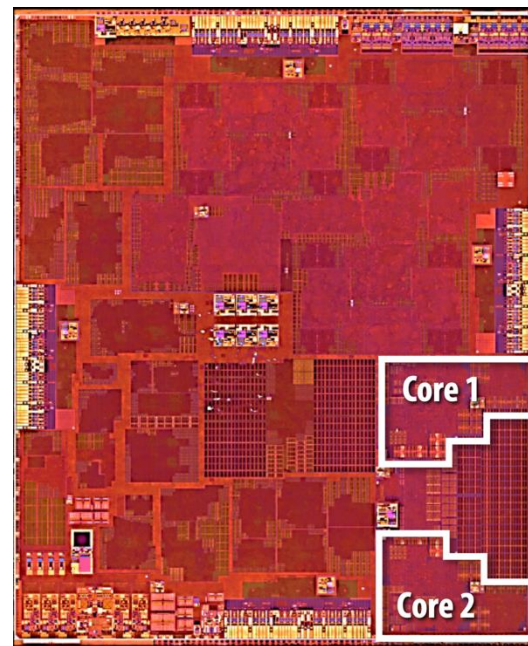
NVIDIA GTX 1080 GPU 20 SM 核心

多核处理器

■ 多核处理器示例-2



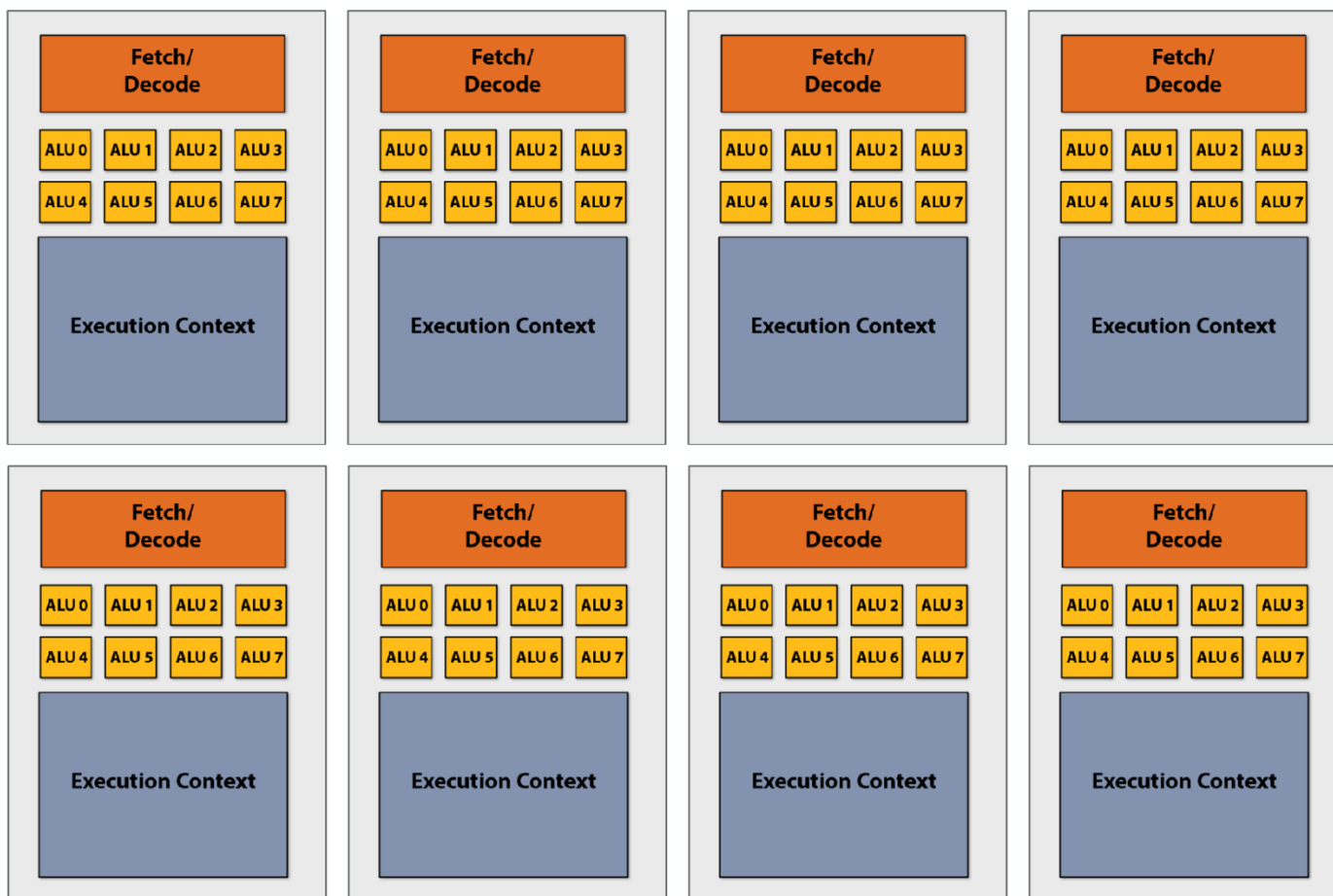
Intel Xeon Phi Knights Corner 72 核处理器



Apple A9 双核处理器

多核处理器

- 多核处理器：
 - 采用多线程或多进程发挥更多处理器核心的计算能力
 - Pthread、OpenMP、MPI 等



总结

- 处理器加速技术：
 - 流水线
 - 超标量
 - SIMD
 - 分支预测
 - 乱序执行
- 存储层次：cache
- 给程序员优化的一些启发：
 - 减少程序分支 → 让流水线尽量一直工作
 - 提供更多的无关指令 → 超标量可以起作用
 - 循环展开 → 提供更多并行机会
 - 数据并发 → 利用 SIMD 优化
 - 访存很慢 → 提高程序的局部性、减少对主存的访问

小作业-5

SIMD 练习:

- 基于 intel intrinsic 实现 $a+b$ 的向量化，与无向量化、编译器自动向量化的代码进行性能对比
- 目的：
 - 学习向量化指令的使用方法，体会向量化带来的性能提升。
- 详见网络学堂